2026

기술소개서

이름 김창홍

전화번호 010-4720-0578

이메일 1138aiie@gmail.com

INDEX

목차

P.3 DirectX11을 이용한 3D 개인 포트폴리오

P.27 DirectX11을 이용한 3D 팀 포트폴리오

P.53 DirectX9을 이용한 2.5D 팀 포트폴리오

P.61 참고문헌

DirectX11을 이용한 3D 개인 포트폴리오

원작게임

Code Vein

작업기간

2025.02.03 ~ 2025.04.25

작업인원

1인 게임 모작

중점 구현 사항

소울 라이크 장르의 게임으로 3D 모델을 출력하는 것과 기본적인 움직임 및 전투를 구현했습니다.

사용 언어

C++
HLSL 5.0

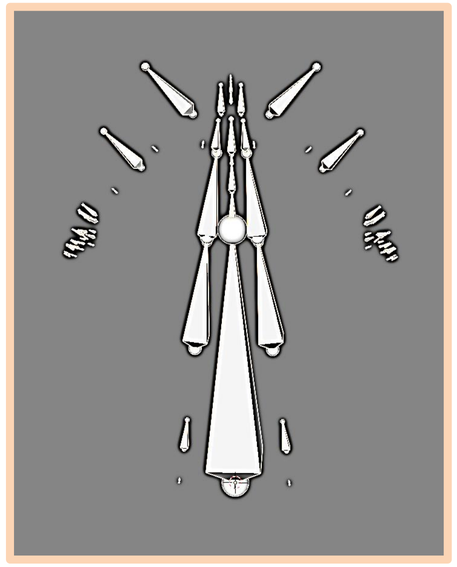
제작 환경

Visual Studio 2022
IMGUI
Assimp

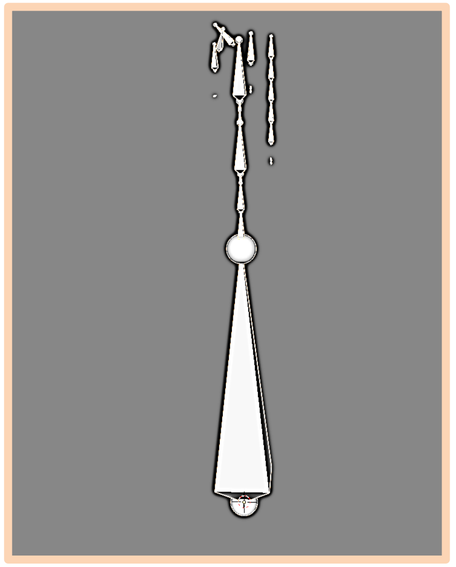
<바디 이미지>

<헤드 이미지>

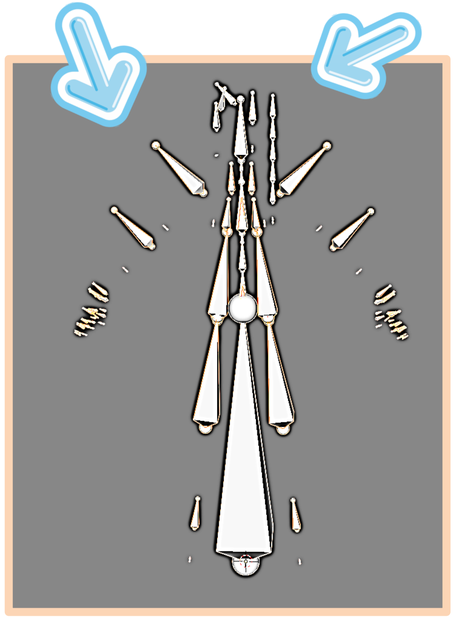
<전체 이미지>



<바디 본 이미지>



<헤드 본 이미지>



<병합 본 이미지>

본 병합

기술 정의

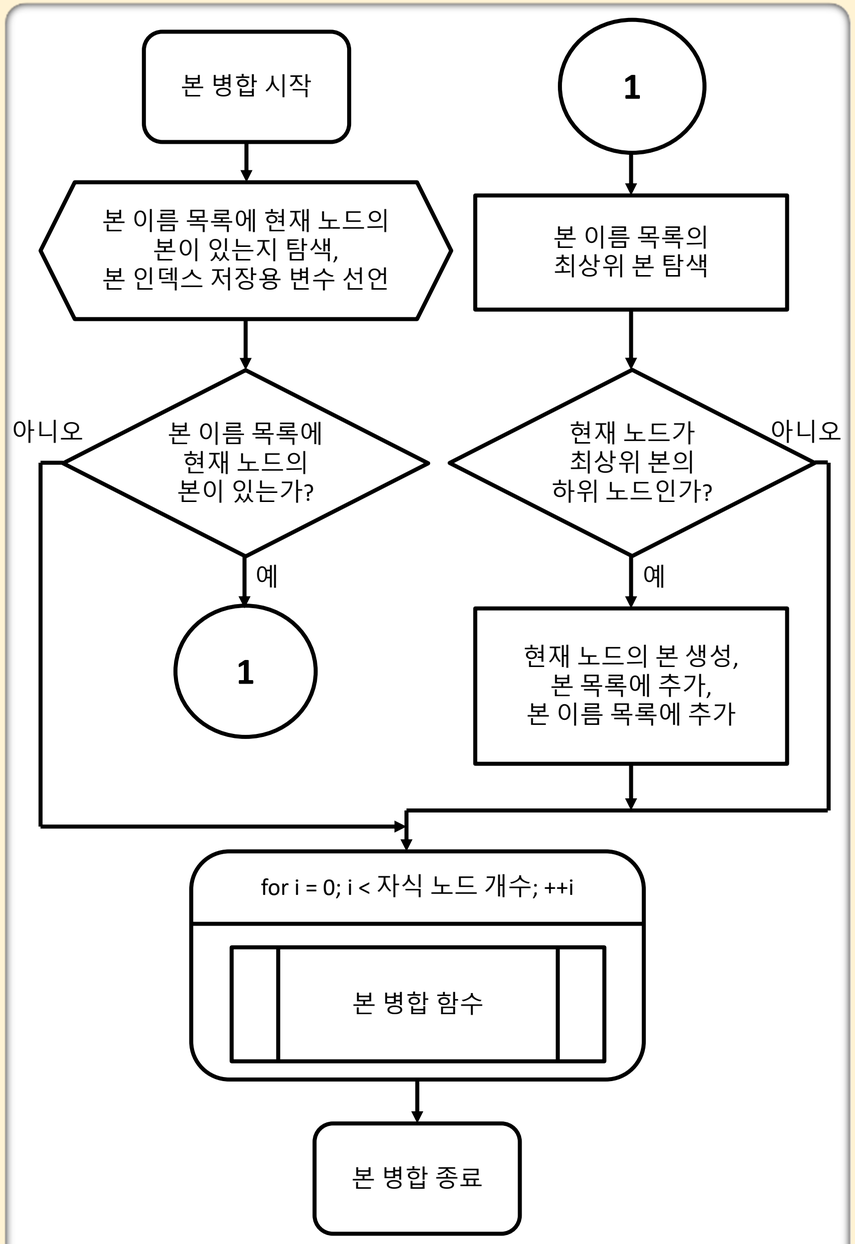
다른 모델들의 공통된 본을 이용해 하나의 연결된 본을 만드는 기술

사용 목적

캐릭터를 구성하는 모델들이 분리되어 있고 본이 연결되지 않아 병합하기 위해 사용했습니다.

구현 방법

캐릭터의 모델을 읽을 때 기존 모델의 포인터를 매개변수로 사용해서 기존 모델의 본 목록을 추가하는 방법으로 구현했습니다.



<본 병합 함수 순서도>

```
HRESULT CModel_Pos::Add_Bones(const aiNode* _pAINode, _int _iParentBoneIndex)
{
    auto Pair = m_BoneNames.find(_pAINode->mName.data);

    _uint iNumBones = { };

    if (Pair == m_BoneNames.end())
    {
        Pair = m_BoneNames.find("CHARA_OFFSET");

        if (_iParentBoneIndex >= static_cast<_int>((*Pair).second))
        {
            CBone* pBone = CBone::Create(_pAINode, _iParentBoneIndex);
            if (nullptr == pBone)
                return E_FAIL;

            m_Bones.push_back(pBone);

            iNumBones = static_cast<_uint>(m_Bones.size());

            m_BoneNames.emplace(_pAINode->mName.data, iNumBones - 1);

            for (size_t i = 0; i < _pAINode->mNumChildren; i++)
                Add_Bones(_pAINode->mChildren[i], iNumBones - 1);
        }
        else
        {
            for (size_t i = 0; i < _pAINode->mNumChildren; i++)
                Add_Bones(_pAINode->mChildren[i], _iParentBoneIndex + 1);
        }
    }
    else
    {
        for (size_t i = 0; i < _pAINode->mNumChildren; i++)
            Add_Bones(_pAINode->mChildren[i], _iParentBoneIndex + 1);
    }

    return S_OK;
}
```

현재 추가하려는 본이 본 이름 목록에 있는지 확인해서 목록에 있다면 추가하지 않습니다.

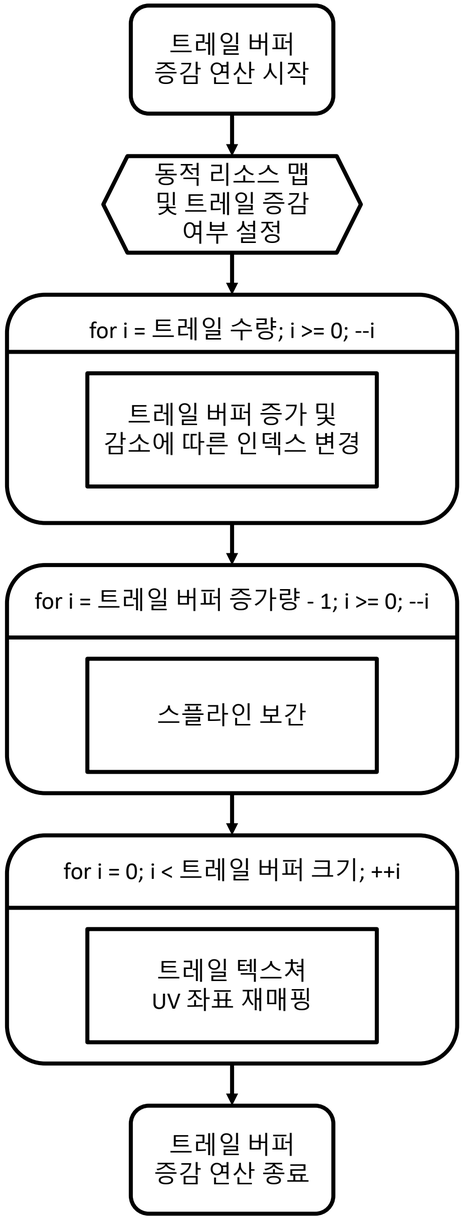
파일 이름으로 된 불필요한 본을 제외하기 위해 실질적인 루트로 사용되는 본인지 확인합니다.

본을 새로 만들고 공통 본에 자식 본을 추가하고 본 인덱스 목록과 본 이름 목록에 새로 만든 본을 추가합니다.

재귀적으로 자식 본들을 순회하며 자식 본이 없을 때 까지 재귀를 반복해서 읽어 들인 파일의 모든 본을 탐색합니다.

<트레일 비활성 이미지 >

<트레일 활성 이미지>



<트레일 함수 순서도>

트레일

기술 정의

행동의 흔적을 남기는 기술

사용 목적

공격 시 흔적이 남지 않아 밋밋함을 느껴
잔상을 남기기 위해 사용했습니다.

구현 방법

공격이 활성화되었을 때 검 끝의 좌표와 검
손잡이 끝의 좌표를 전달해서 버퍼에 추가

```
void CVIBuffer_Line_Trail::Trail(_float _fTimeDelta, _bool _bIncrease, _vector _vPommel,
_vector _vBlade)
{
// 중략
    _uint iDecrease = { };
    if (_bIncrease == true)
    {
        if (m_iNumInstances == 0)
        {
            pParticles[0].vLifeTime.y = 0.f;
            XMStoreFloat4(&pMatrices[0].vRight, _vPommel);
            XMStoreFloat4(&pMatrices[0].vUp, _vBlade);
            XMStoreFloat4(&pMatrices[0].vLook, _vPommel);
            XMStoreFloat4(&pMatrices[0].vTranslation, _vBlade);
            XMStoreFloat4(&m_vPreInterpolationPommel, _vPommel);
            XMStoreFloat4(&m_vPreInterpolationBlade, _vBlade);
            pMatrices[0].vTexcoord = _float2(0.f, 1.f);

            ++m_iNumInstances;
        }
        else
        {
            _uint iIncrease = { 1 };
            _float fLength =
XMVectorGetX(XMVector3Length(XMLoadFloat4(&pMatrices[0].vTranslation) - _vBlade));

            if (fLength > m_fThreshold)
            {
                _uint iImpolationIncrease = static_cast<_uint>(fLength / m_fThreshold);
                if (m_iNumInstances + iImpolationIncrease < m_iMaxInstance)
                    iIncrease += iImpolationIncrease;
            }

            if (m_iNumInstances + iIncrease < m_iMaxInstance)
            {
                Trail_Traversal(pMatrices, pParticles, static_cast<_int>(m_iNumInstances
- 1), iIncrease, iDecrease, _fTimeDelta);
            }
            else
            {
                Trail_Traversal(pMatrices, pParticles, static_cast<_int>(m_iMaxInstance
- iIncrease - 1), iIncrease, iDecrease, _fTimeDelta);
            }
```

트레일이 버퍼가 비어있다가
증가하는 경우라면
전달받은 좌표로
트레일을 추가하고
트레일 텍스쳐 UV좌표를
맵핑합니다.

이전 트레일과
거리가 너무 먼 경우
스플라인 보간을 하더라도
끊겨 보이기 때문에
트레일 간격이 너무 넓다면
중간에 트레일 지점을
추가합니다.

트레일을 순회하며
버퍼의 인덱스를 관리합니다

```
void CVIBuffer_Line_Trail::Trail_Traversal(VTXINSTANCETRAIL* _pMatrices,
VTXPARTICLE* _pParticles, _int _iStartIndex, _int _iIncrease, _uint&
_refDecrease, _float _fTimeDelta)
{
    for (_int i = _iStartIndex; i >= 0; --i)
    {
        _pParticles[i].vLifeTime.y += _fTimeDelta;
        if (_pParticles[i].vLifeTime.y >= _pParticles[i].vLifeTime.x)
        {
            _pParticles[i].vLifeTime.y = _pParticles[i].vLifeTime.x + 1.f;
            _pMatrices[i].vRight = _float4(1.f, 0.f, 0.f, 0.f);
            _pMatrices[i].vUp = _float4(0.f, 1.f, 0.f, 0.f);
            _pMatrices[i].vLook = _float4(0.f, 0.f, 1.f, 0.f);
            _pMatrices[i].vTranslation = _float4(0.f, 0.f, 0.f, 1.f);
            ++_refDecrease;
        }
        else
        {
            _pParticles[i + _iIncrease].vLifeTime.y = _pParticles[i].vLifeTime.y;
            _pMatrices[i + _iIncrease].vRight = _pMatrices[i].vRight;
            _pMatrices[i + _iIncrease].vUp = _pMatrices[i].vUp;
            _pMatrices[i + _iIncrease].vLook = _pMatrices[i].vLook;
            _pMatrices[i + _iIncrease].vTranslation = _pMatrices[i].vTranslation;
        }
    }
}
```

트레일 버퍼를 순회하며
생명주기가 끝난 인덱스에는
버퍼의 초기값을 대입하고,
추가되는 트레일이 배열의 앞에 배치되도록
생명주기가 남아있는 기존의 트레일은
추가되는 버퍼만큼 뒤로 밀어서
버퍼의 인덱스를 변경합니다.

```
            if (iIncrease > 1)
            {
                _float fRatio = 1.f / static_cast<_float>(iIncrease);

                for (_int i = static_cast<_int>(iIncrease - 1); i >= 0; --i)
                {
                    _vector vInterpolationPommel = { }, vInterpolationBlade = { };

                    vInterpolationPommel = CatmullRom(_vPommel, _vPommel,
XMLoadFloat4(&pMatrices[iIncrease].vLook), XMLoadFloat4(&m_vPreInterpolationPommel),
fRatio* i);
                    vInterpolationBlade = CatmullRom(_vBlade, _vBlade,
XMLoadFloat4(&pMatrices[iIncrease].vTranslation),
XMLoadFloat4(&m_vPreInterpolationBlade), fRatio* i);

                    pParticles[i].vLifeTime.y = 0.f;
                    pMatrices[i].vRight = pMatrices[i + 1].vLook;
                    pMatrices[i].vUp = pMatrices[i + 1].vTranslation;
                    XMStoreFloat4(&pMatrices[i].vLook, vInterpolationPommel);
                    XMStoreFloat4(&pMatrices[i].vTranslation, vInterpolationBlade);
                }

                m_vPreInterpolationPommel = pMatrices[iIncrease].vLook;
                m_vPreInterpolationBlade = pMatrices[iIncrease].vTranslation;
            }
            else
            {
                pParticles[0].vLifeTime.y = 0.f;
                pMatrices[0].vRight = pMatrices[1].vLook;
                pMatrices[0].vUp = pMatrices[1].vTranslation;
                XMStoreFloat4(&pMatrices[0].vLook, _vPommel);
                XMStoreFloat4(&pMatrices[0].vTranslation, _vBlade);
            }
            m_iNumInstances += iIncrease - iDecrease;
            Trail_RemapUV(pMatrices);
        }
    }
    else
    {
        for (_uint i = 0; i < m_iNumInstances; ++i)
        {
            pParticles[i].vLifeTime.y += _fTimeDelta;
            if (pParticles[i].vLifeTime.y >= pParticles[i].vLifeTime.x)
            {
                Trail_Decrease(pMatrices, pParticles, i);
                ++iDecrease;
            }
        }
        m_iNumInstances -= iDecrease;
        Trail_RemapUV(pMatrices);
    }
    m_pContext->Unmap(m_pVBParticle, 0);
    m_pContext->Unmap(m_pVBTrail, 0);
}
```

1

추가된 트레일을 순회하며 스플라인 보간을 적용해서 트레일이 완만한 곡선을 이루게 합니다.

2

```
_vector CVIBuffer_Line_Trail::CatmullRom(const _fvector& _vNewPos0, const
_fvector& _NewPos1, const _fvector& _vOldPos0, const _gvector& _vOldPos2, _float
_fRatio)
{
    return 0.5f * ((2.0f * _NewPos1) +
        (-_vNewPos0 + _vOldPos0) * _fRatio +
        (2.0f * _vNewPos0 - 5.0f * _NewPos1 + 4.0f * _vOldPos0 - _vOldPos2) *
(_fRatio * _fRatio) +
        (-_vNewPos0 + 3.0f * _NewPos1 - 3.0f * _vOldPos0 + _vOldPos2) * (_fRatio
* _fRatio * _fRatio));
}
```

2

현재 검 손잡이 끝의 좌표와 이전 검 손잡이 끝의 좌표를 스플라인 보간하고 보간한 구간을 n등분해서 중간 좌표들을 각 i번째 트레일 위치로 반환합니다.

3

트레일이 시작되는 지점의 두 좌표를 이전 보간 종료 지점으로 저장해서 이후 트레일을 보간할 때 순회 범위를 좁힐 수 있도록 했습니다.

4

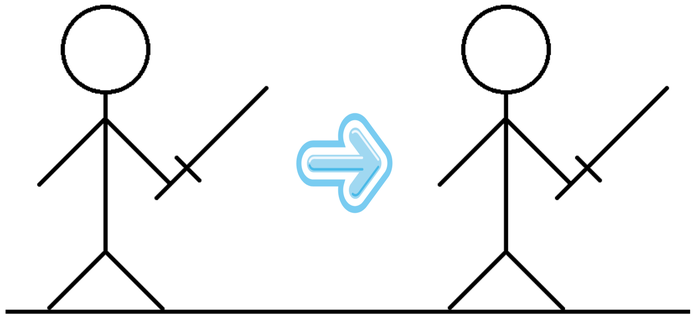
현재 트레일 버퍼의 크기를 확정짓고, 트레일 버퍼에 트레일 텍스쳐의 UV 좌표를 다시 맵핑합니다.

4

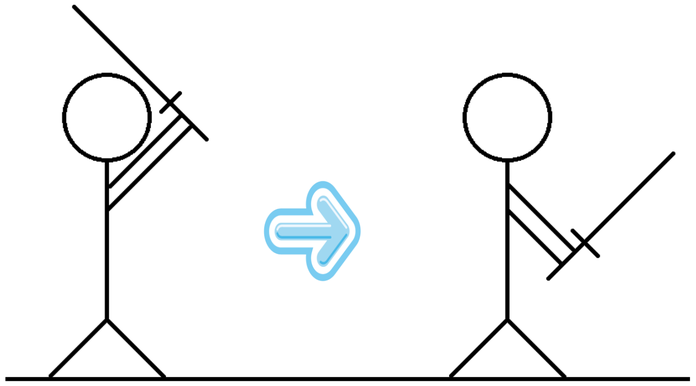
```
    for (_uint i = 0; i < m_iNumInstances; ++i)
    {
        XMStoreFloat2(&_pMatrices[i].vTexcoord,
XMVectorSet((static_cast<_float>(i) / m_iNumInstances),
(static_cast<_float>(i + 1) / m_iNumInstances), 0.f, 0.f));
    }
```

4

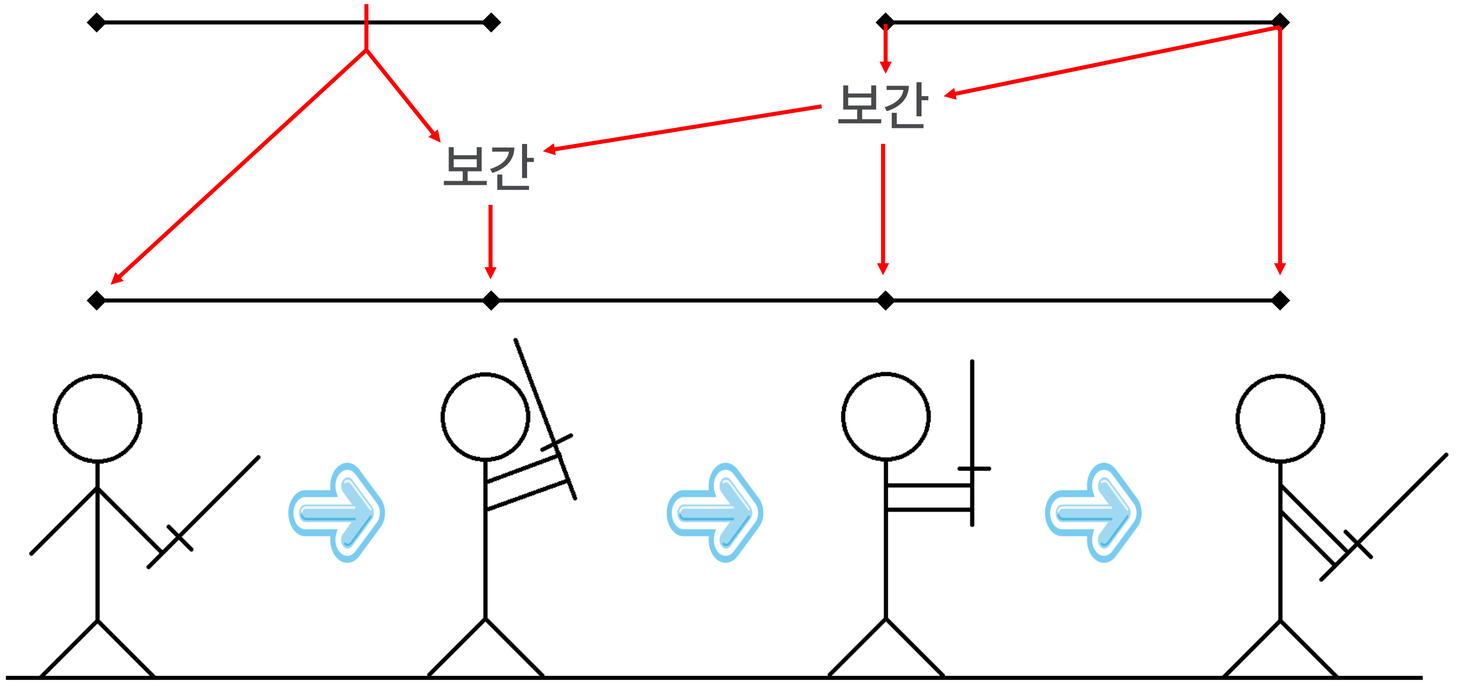
텍스쳐 UV 좌표를 트레일 버퍼의 크기만큼 나누고 트레일과 UV 좌표를 동일 순서로 맵핑합니다.



<대기 모션 키 프레임 예시>



<공격 모션 키 프레임 예시>



<대기 모션에서 공격 모션으로 전환 시 선형 보간 예시>

애니메이션 전환 시 선형보간

기술 정의

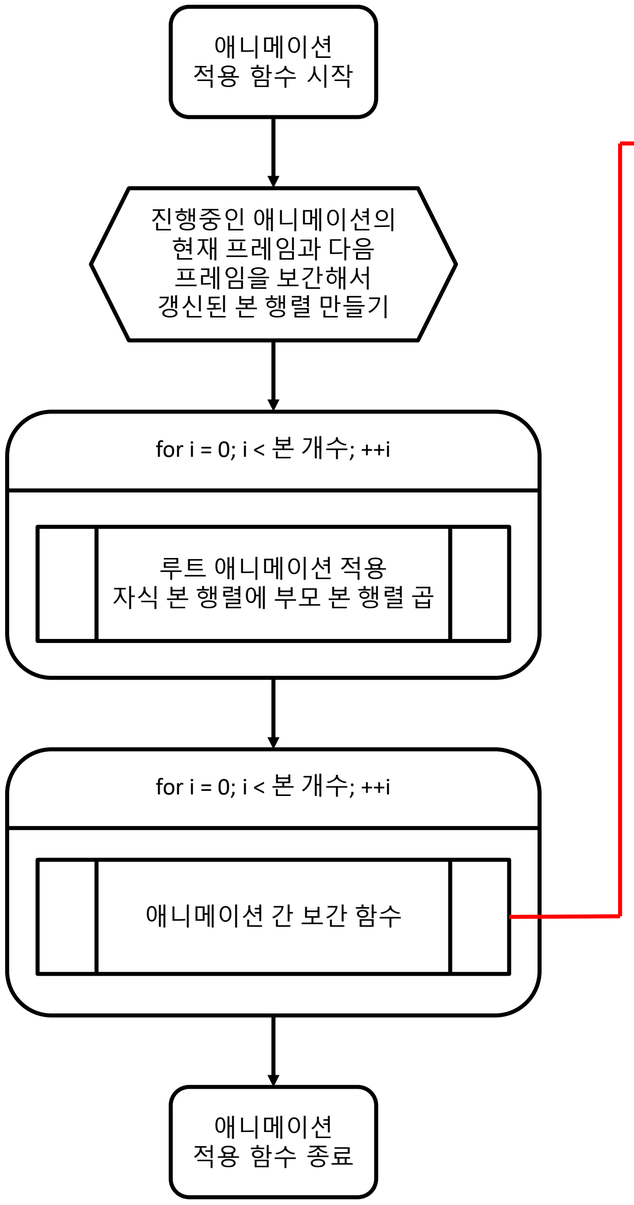
끝점의 값이 주어졌을 때 직선거리에 따라
선형적으로 그 사이에 위치한 값을 추정하는 기술

사용 목적

애니메이션 전환 시 모션이 끊겨 보이지 않고
자연스럽게 보이기 위해 사용했습니다.

구현 방법

애니메이션이 전환될 때 현재 본들의 행렬을
저장하고 전환된 애니메이션의 한 키 프레임이
진행되는 동안 현재 본들의 행렬과 저장된 본들의
행렬을 보간했습니다.



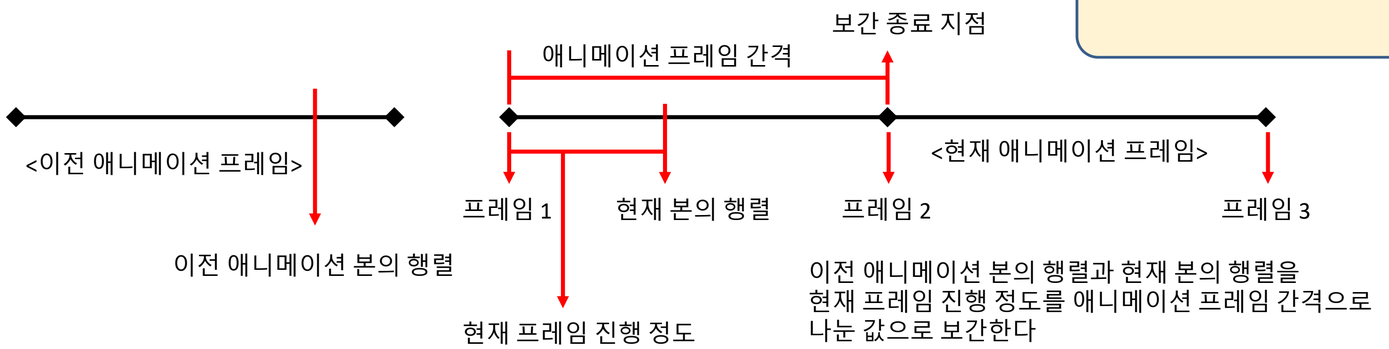
<애니메이션 적용 함수 순서도>

```
void CBone::Update_RootInterpolationCombinedTransformationMatrix(const
vector<CBone*>& _Bones, const _float* _pCurrentTrackPosition, const _float*
_pInterpolationLength, const _float4x4* _pInterpolationMatrix)
{
    _vector vScale = { }, vRotation = { }, vTranslation = { };

    _float fRatio = (*_pCurrentTrackPosition / *_pInterpolationLength);

    _vector vSourScale = { }, vDestScale = { };
    _vector vSourRotation = { }, vDestRotation = { };
    _vector vSourTranslation = { }, vDestTranslation = { };

    XMMatrixDecompose(&vSourScale, &vSourRotation, &vSourTranslation,
XMLoadFloat4x4(_pInterpolationMatrix));
    XMMatrixDecompose(&vDestScale, &vDestRotation, &vDestTranslation,
XMLoadFloat4x4(&m_CombinedTransformationMatrix));

    vScale = XMVectorLerp(vSourScale, vDestScale, fRatio);
    vRotation = XMQuaternionSlerp(vSourRotation, vDestRotation, fRatio);
    vTranslation = XMVectorLerp(vSourTranslation, vDestTranslation, fRatio);


    _matrix TransformationMatrix = XMMatrixAffineTransformation(vScale
        , XMVectorSet(0.f, 0.f, 0.f, 1.f), vRotation, vTranslation);

    XMStoreFloat4x4(&m_CombinedTransformationMatrix, TransformationMatrix);
}
```

현재 프레임의 진행 정도를
애니메이션의 프레임 간격으로
나눠 보간율을 구합니다.

본의 이전 애니메이션 행렬과
현재 행렬을 크기 벡터,
회전 벡터, 위치 벡터로 분해합니다.

현재 본과 이전 본의
크기 벡터, 회전 벡터,
위치 벡터를 앞에서 구한
보간율을 이용해서 보간하고
보간된 본 행렬을 만듭니다.



<대쉬 애니메이션>

<루트 애니메이션 미적용 시 대쉬 후 위치>

<루트 애니메이션 적용 시 대쉬 후 위치>

루트애니메이션

기술 정의

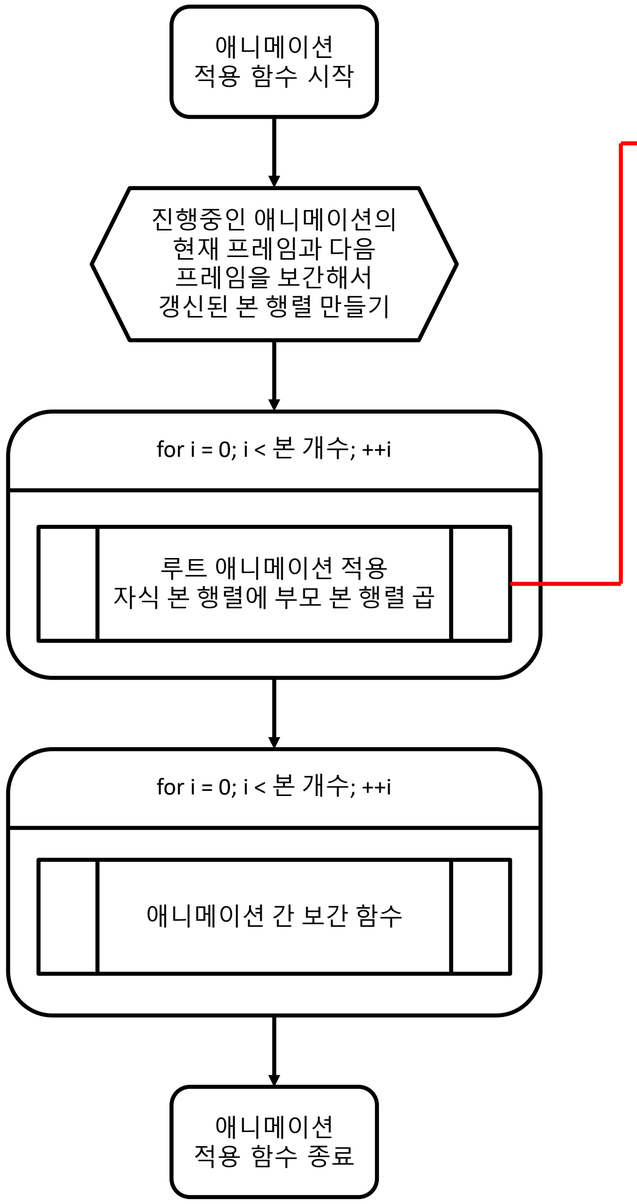
애니메이션의 로컬 이동량을 월드 이동량으로 적용하는 기술

사용 목적

애니메이션에 따른 캐릭터의 이동과 실제 월드상 위치가 불일치하는 문제가 있어 일치시키기 위해 사용했습니다.

구현 방법

로컬의 이동을 저장하는 본의 로컬 위치를 저장하고 다음 업데이트에 이동을 저장하는 본의 로컬 위치 차이를 월드 행렬에 전달한 후 이동을 저장하는 본의 위치를 원점으로 변경해서 사용했습니다.



<애니메이션 적용 함수 순서도>

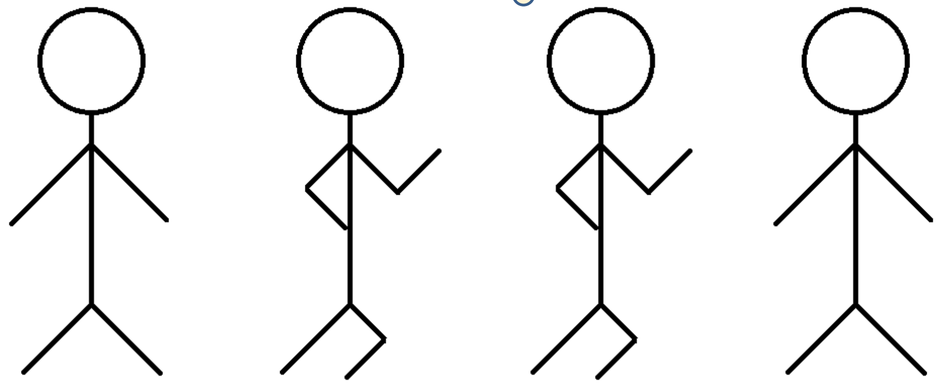
```
void CBone::Update_RootCombinedTransformationMatrix(const
vector<CBone*>& _Bones, _float4* _pTransfer, _float4*
_pPreTranslation)
{
    XMStoreFloat4x4(&m_CombinedTransformationMatrix
        , XMLoadFloat4x4(&m_TransformationMatrix)
        * XMLoadFloat4x4(&_Bones[m_iParentBoneIndex]-
>m_CombinedTransformationMatrix));

    _float3 vZero = { 0.f, 0.f, 0.f };
    _float4 vPosition = { };

    memcpy(&vPosition,
m_CombinedTransformationMatrix.m[3], sizeof(_float3));

    XMStoreFloat4(_pTransfer,(XMLoadFloat4(&vPosition) -
XMLoadFloat4(_pPreTranslation)));

    memcpy(_pPreTranslation,
m_CombinedTransformationMatrix.m[3], sizeof(_float3));

    memcpy(m_CombinedTransformationMatrix.m[3], &vZero,
sizeof(_float3));
}
```

현재 본에 부모 본의 행렬값을 곱해서
부모 본에 따른 크기, 회전, 이동 값을
적용합니다.

루트 애니메이션이 미적용된
현재 본의 로컬 이동 값을 저장합니다.

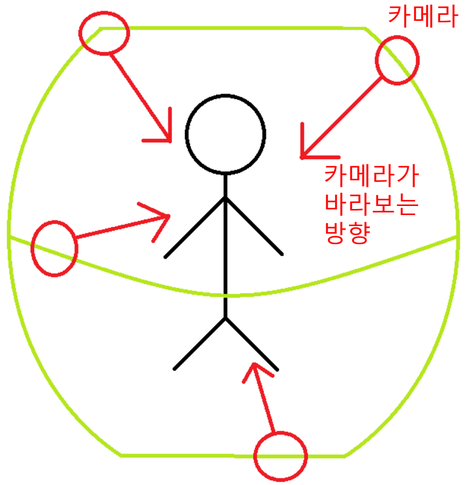
현재 프레임과 이전 프레임의
로컬 이동량의 차이를
월드로 전달할 변수에 저장합니다.

현재 로컬 위치를 이전 로컬 위치에 저장하고
로컬 이동량을 0으로 만듭니다.



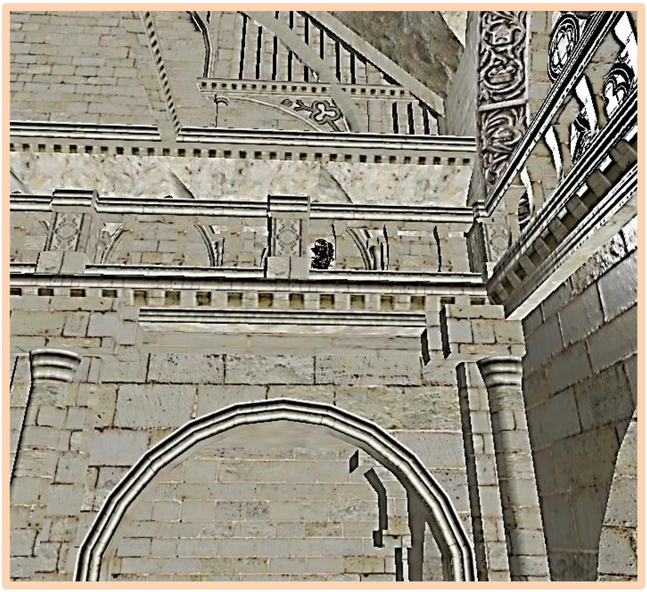
<대쉬 애니메이션>

루트 애니메이션 적용, 미적용 시 월드 좌표 예시				
미적용	(0,0)	(0,0)	(0,0)	(0,0)
적용	(0,0)	(1,0)	(2,0)	(3,0)



<구면 좌표 카메라 예시>

<구면 좌표 카메라 예시1>



<구면 좌표 카메라 예시2>

<구면 좌표 카메라 예시3>

구면 좌표 카메라

기술 정의

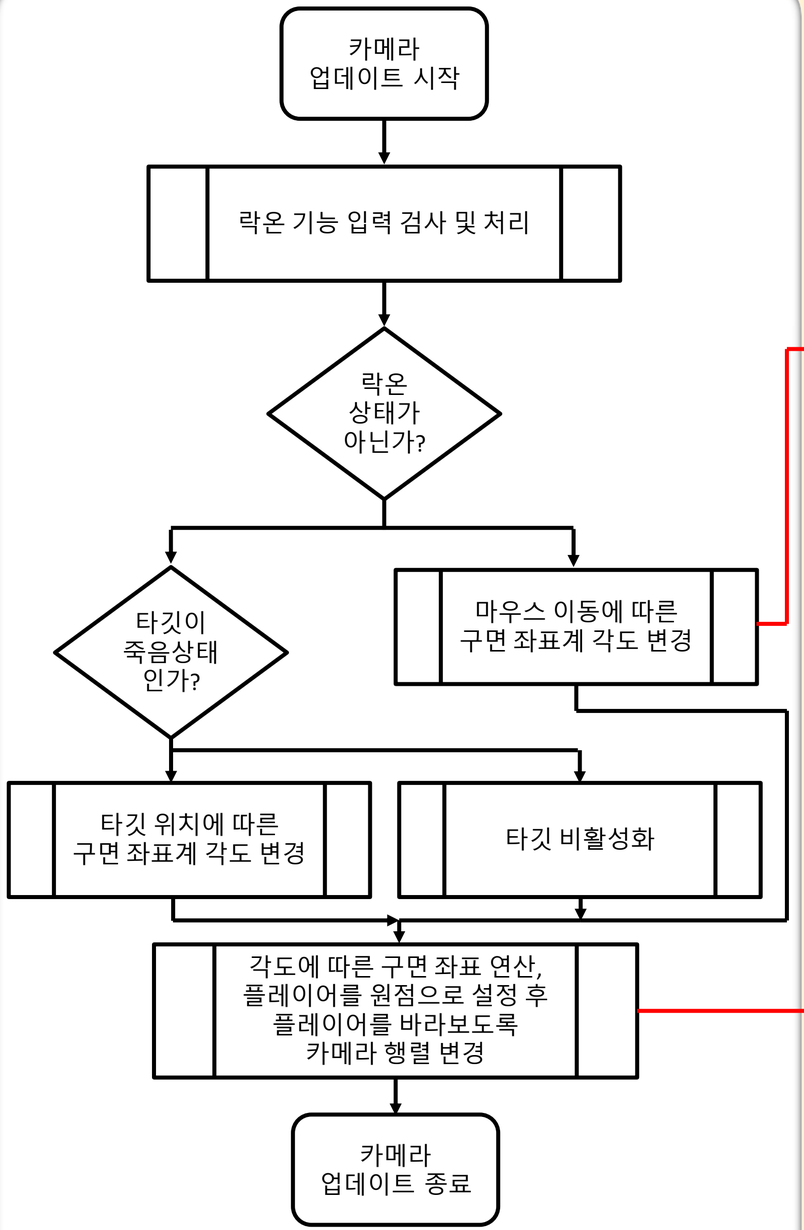
구면 좌표를 따라 이동하는 카메라

사용 목적

카메라가 항상 플레이어를 담고 자유롭게
돌아가기 위해 사용했습니다.

구현 방법

마우스의 수평 이동 시 방위각을 변경했고,
마우스의 수직 이동 시 고각을 변경했습니다.
마우스 휠이 돌아가면 카메라에서 플레이어
까지의 거리를 변경했습니다.



<구면 좌표 카메라 순서도>

```
void CCamera_Player::Priority_Update(_float _fTimeDelta)
{
    // 중략

            if (_float fZ = static_cast<_float>(m_pGameInstance->Get_DIMouseMove(MOUSEMOVESTATE::DIMS_Z)))
            {
                m_fRadius += -fZ * _fTimeDelta * 0.1f;
                m_fRadius = clamp(m_fRadius, 1.f, 5.5f);
            }

            _long        MouseMove{};

            if (MouseMove = m_pGameInstance->Get_DIMouseMove(MOUSEMOVESTATE::DIMS_X))
            {
                m_fAzimuth += MouseMove * m_pTransformCom->Get_Speed() * 0.001f;
                if (m_fAzimuth >= 6.28f)
                    m_fAzimuth = fmodf(m_fAzimuth, 6.28f);
            }

            if (MouseMove = m_pGameInstance->Get_DIMouseMove(MOUSEMOVESTATE::DIMS_Y))
            {
                m_fElevation += -MouseMove * m_pTransformCom->Get_Speed() * 0.001f;
                m_fElevation = clamp(m_fElevation, 0.01f, 3.14f);
            }

    // 중략

    if (m_fAzimuth < 0.f)
        m_fAzimuth = XMConvertToRadians(360.f) + m_fAzimuth;
    else if (m_fAzimuth >= XMConvertToRadians(360.f))
        m_fAzimuth = 0.f;
    m_fElevation = clamp(m_fElevation, 0.01f, 3.14f);

    XMStoreFloat4(&m_vSphericalCoordinate, XMVectorSet(
        m_fRadius * sin(m_fElevation) * sin(m_fAzimuth),
        m_fRadius * cos(m_fElevation),
        m_fRadius * sin(m_fElevation) * cos(m_fAzimuth),
        0.f));

    m_pTransformCom->Set_State(CTransform::STATE_POSITION, XMLoadFloat4x4(m_pPlayerTransformMatrix).r[3] + XMLoadFloat4(&m_vSphericalCoordinate));

    if (m_pTargetObject == nullptr)
        m_pTransformCom->LookAt(XMLoadFloat4x4(m_pPlayerTransformMatrix).r[3] + XMVectorSet(0.f, 1.f, 0.f, 0.f));
    else
        m_pTransformCom->LookAt(static_cast<CTransform*>(m_pTargetObject->Get_Component(TEXT("Com_Transform")))->Get_State(CTransform::STATE_POSITION));

    // 중략

    __super::Renew_Matrices();
}
```

카메라와 플레이어 간의 거리를 조절합니다.

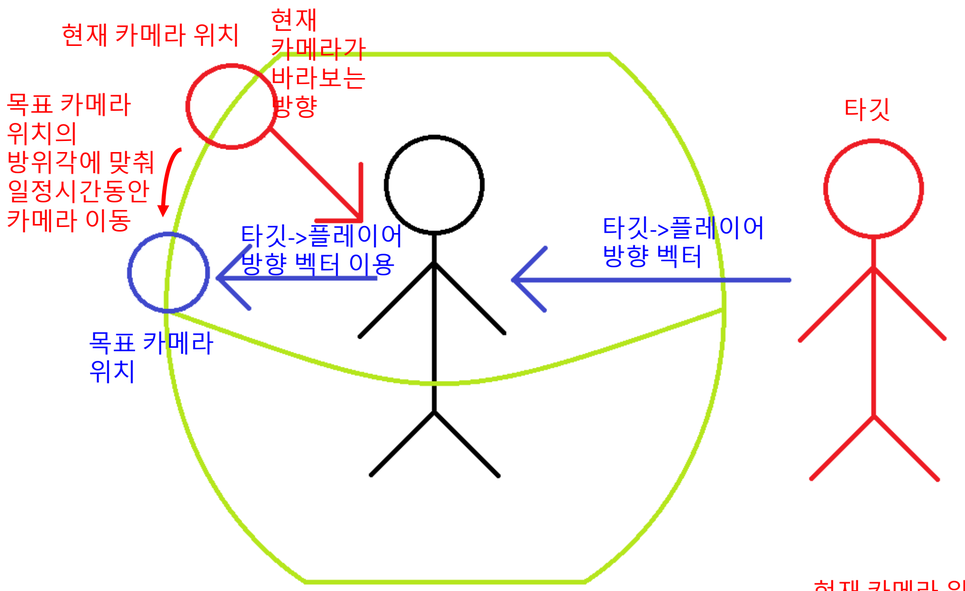
마우스의 수평 이동에 따라 방위각의 값을 조절하고
방위각이 0과 최대 각도 사의 값을 유지하도록 합니다.

마우스의 수직 이동에 따라 고각의 값을 조절하고 한계값을 넘지 않도록 고각의 값을 제한합니다.

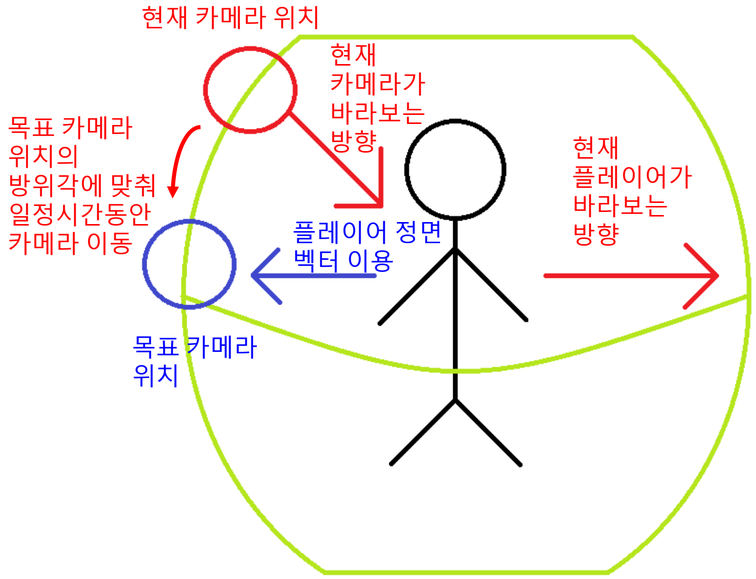
방위각과 고각에 따른 구면 좌표를 구합니다.

플레이어의 좌표에 구면 좌표를 더해서 플레이어의 좌표를 원점으로 사용합니다.

카메라가 플레이어의 허리를 바라보도록 목표 지점을 설정합니다.



<타깃이 있을 때 락온 기능 예시>



<타깃이 없을 때 락온 기능 예시>

카메라 락온

기술 정의

타깃을 고정하고 항상 해당 타깃을 바라보는 카메라 기능

사용 목적

전투 시 고정한 몬스터를 계속 바라보도록 하기 위해 사용했습니다.

구현 방법

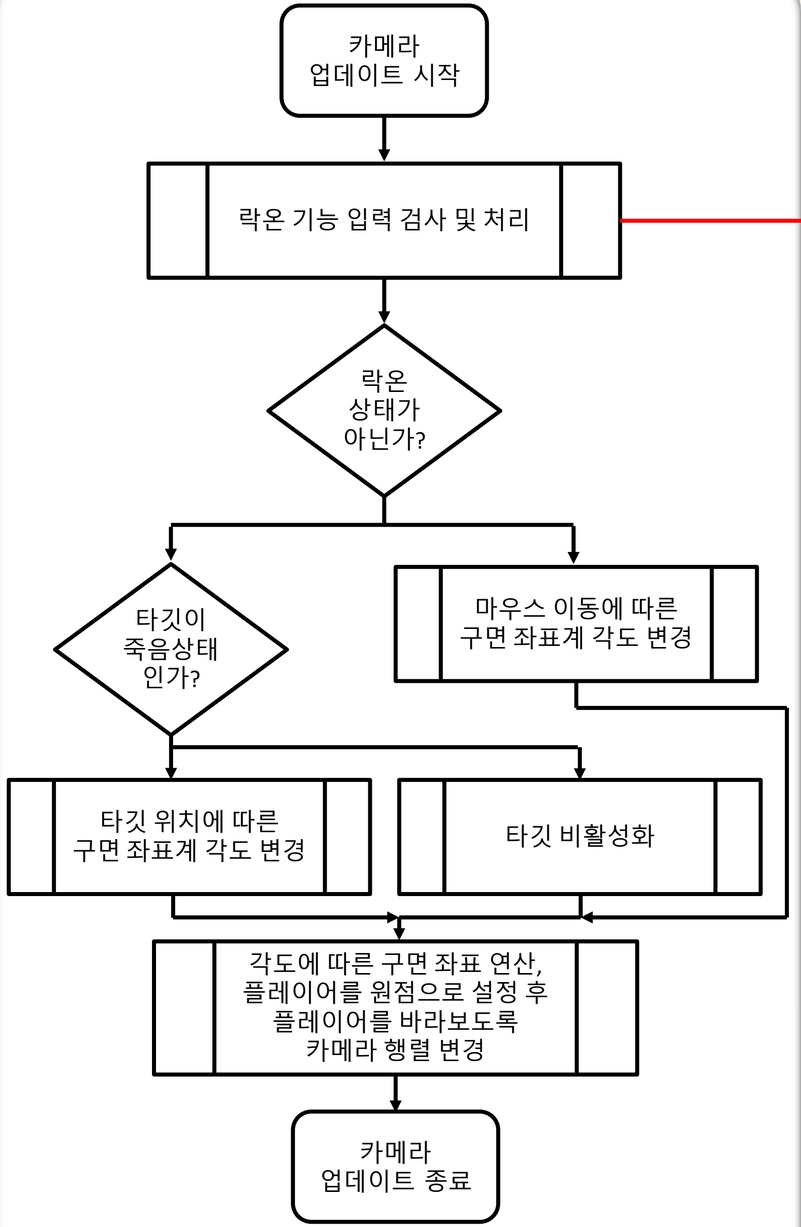
플레이어를 인식한 몬스터 중 카메라가 바라보는 방향의 벡터를 연장했을 때 일정 범위 내에 있는 몬스터의 포인터를 저장해두고 락온 중 해당 몬스터의 좌표로 카메라가 바라보는 지점을 갱신해서 구현했습니다.

타깃이 있을 때 락온한 경우

카메라 → 플레이어 → 타깃이 일직선 상에 놓입니다.

타깃이 없을 때 락온한 경우

카메라가 바라보는 방향이 플레이어가 바라보는 방향과 일치할 때까지 이동합니다.



<구면 좌표 카메라 순서도>

```
void CCamera_Player::Priority_Update(_float _fTimeDelta)
{
    if (m_bActive == false)
        return;

1   if (m_pGameInstance->Get_DIMouseInputState(MOUSEKEYSTATE::DIM_MB)
 == MOUSEINPUTSTATE::DIMI_MBD)
    {
2       CPlayer* pPlayer = static_cast<CPlayer*>(m_pGameInstance-
>Get_GameObject(LEVEL_GAMEPLAY, TEXT("Layer_Player"), 0));
        if (pPlayer->Check_Collision() == true)
        {
6           IGameObject* pTargetObject = pPlayer-
>Target_LockOn(m_pTransformCom->Get_State(CTransform::STATE_LOOK));

            if (pTargetObject == nullptr)
            {
                if (m_pTargetObject != nullptr)
                {
                    Safe_Release(m_pTargetObject);
                    m_pTargetObject = nullptr;
                    pPlayer->Release_Target();
                }
                else
3                   Target_None();
            }
            else
            {
                if (m_pTargetObject != pTargetObject)
                {
4                   Safe_Release(m_pTargetObject);
                    m_pTargetObject = pTargetObject;
                    Safe_AddRef(m_pTargetObject);
                    pPlayer->Set_Target(m_pTargetObject);
                }
                else
5               {
                    Safe_Release(m_pTargetObject);
                    m_pTargetObject = nullptr;
                    pPlayer->Release_Target();
                }
            }
        }
        else
            Target_None();
    }
```

1. 락온 키 입력을 확인합니다.
2. 플레이어의 포인터를 얻고 플레이어를 인식한 몬스터가 있는지 확인합니다.
6. To P.19 - 1

3.
```
void CCamera_Player::Target_None()
{
    m_bPlayerLook = true;
    _vector vPlayerLook =
XMVectorSetW((XMVector3Normalize(XMVectorSetY(XMLoadFloat4x4(
m_pPlayerTransformMatrix).r[2], 0.f))), 0.f);
    m_fTargetAzimuth = XMConvertToRadians(180.f) +
atan2f(XMVectorGetX(XMVector3Dot(XMVectorSet(0.f, 1.f, 0.f, 0.f),
XMVector3Cross(XMVectorSet(0.f, 0.f, 1.f, 0.f), vPlayerLook)))
        , XMVectorGetX(XMVector3Dot(XMVectorSet(0.f, 0.f, 1.f, 0.f),
vPlayerLook)));
    m_fTargetElevation = 1.3f;
}
```

플레이어가 바라보는 방향의 방위각을 구해서 저장하고
고각을 플레이어의 높이에 맞춰 변경합니다.

4. 타깃을 변경합니다.
5. 락온을 해제합니다.

```
    // 중략
    else if (m_pTargetObject->Get_Dead() == true)
    {
        Safe_Release(m_pTargetObject);
        m_pTargetObject = nullptr;
        static_cast<CPlayer*>(m_pGameInstance->Get_GameObject(LEVEL_GAMEPLAY,
TEXT("Layer_Player"), 0))->Release_Target();
    }
    else
    {
        _vector vEnemyPos = static_cast<CTransform*>(m_pTargetObject-
>Get_Component(TEXT("Com_Transform")))->Get_State(CTransform::STATE_POSITION);
        _vector vTargetDirection = XMVectorSetW((XMVector3Normalize(XMVectorSetY(vEnemyPos -
XMLoadFloat4x4(m_pPlayerTransformMatrix).r[3], 0.f))), 0.f);
        _float fAzimuth = XMConvertToRadians(180.f) +
atan2f(XMVectorGetX(XMVector3Dot(XMVectorSet(0.f, 1.f, 0.f, 0.f),
XMVector3Cross(XMVectorSet(0.f, 0.f, 1.f, 0.f), vTargetDirection)))
            , XMVectorGetX(XMVector3Dot(XMVectorSet(0.f, 0.f, 1.f, 0.f), vTargetDirection)));

        if (fabsf(m_fAzimuth - fAzimuth) > m_pTransformCom->Get_Speed() * 0.03f)
        {
            if (fabsf(m_fAzimuth - fAzimuth) > XMConvertToRadians(315.f))
                m_fAzimuth = fAzimuth;
            else
            {
                fAzimuth = fAzimuth - m_fAzimuth;
                m_fAzimuth += fAzimuth * m_pTransformCom->Get_Speed() * 0.03f;
            }
        }
        else
            m_fAzimuth = fAzimuth;

        if (fabsf(m_fElevation - 1.3f) >= m_pTransformCom->Get_Speed() * 0.1f)
        {
            if (m_fElevation - 1.3f < 0.f)
                m_fElevation += m_pTransformCom->Get_Speed() * 0.01f;
            else
                m_fElevation += m_pTransformCom->Get_Speed() * -0.01f;
        }
        else
            m_fElevation = 1.3f;
    }

    // 중략

    if (m_pTargetObject == nullptr)
        m_pTransformCom->LookAt(XMLoadFloat4x4(m_pPlayerTransformMatrix).r[3] +
XMVectorSet(0.f, 1.f, 0.f, 0.f));
    else
        m_pTransformCom->LookAt(static_cast<CTransform*>(m_pTargetObject-
>Get_Component(TEXT("Com_Transform")))->Get_State(CTransform::STATE_POSITION));

    // 중략

    __super::Renew_Matrices();
}
```

1 FROM P.18 - 1

```
IGameObject* pTargetObject = pPlayer-
>Target_LockOn(m_pTransformCom->Get_State(CTransform::STATE_LOOK));
```

```
IGameObject* CPlayer::Target_LockOn(_vector _vLookDirection)
{
    return m_pColliderCom->Target_LockOn(m_pTransformCom-
>Get_State(CTransform::STATE_POSITION), _vLookDirection);
}
```

1 가장 가까운 타깃을 구하기 위해 변수를 정의하고 임의의 큰 값으로 거리를 초기화합니다.

2 리스트를 순회하며 플레이어 정면 15도 내의 가장 가까운 타깃을 탐색합니다.

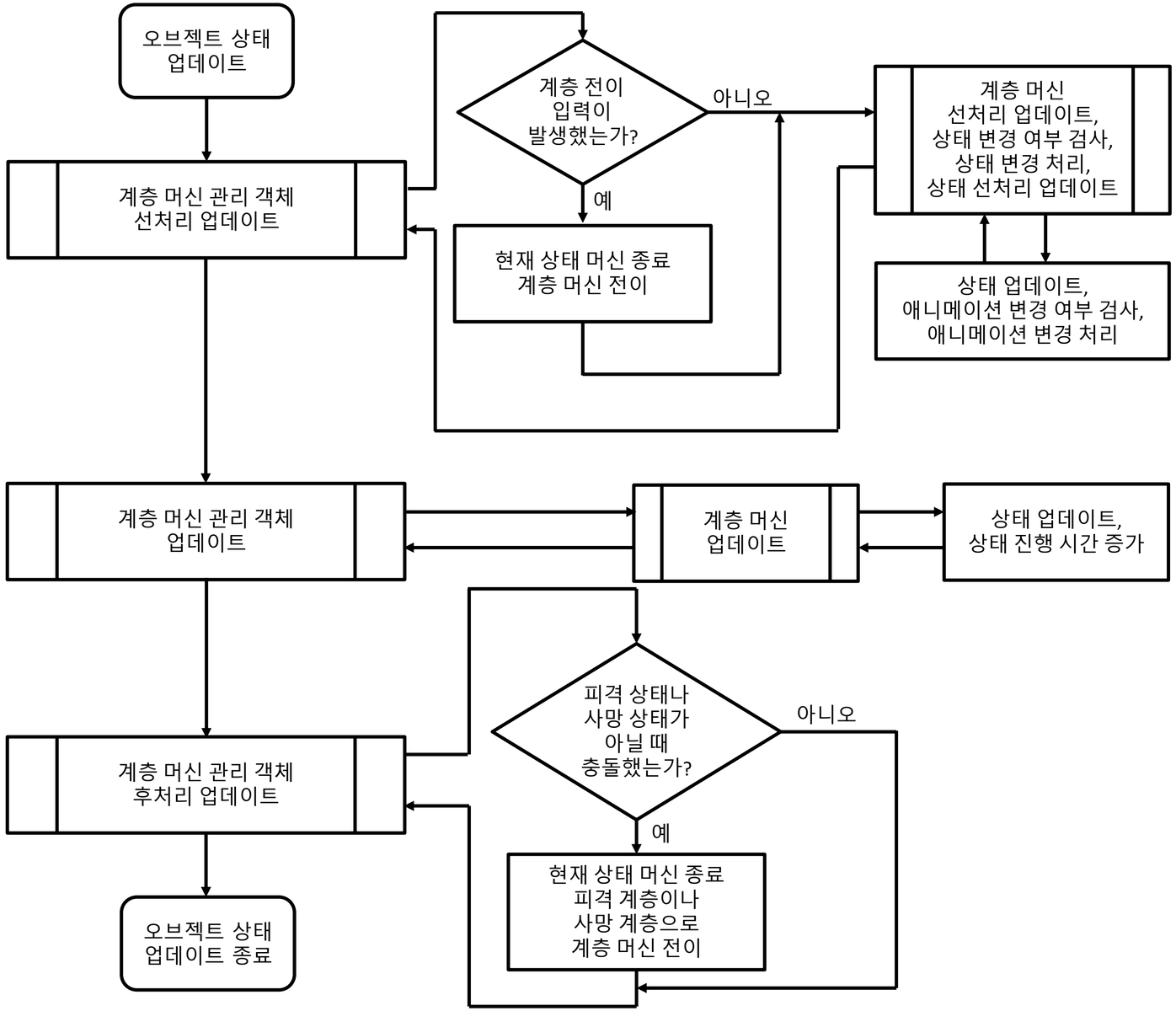
3 타깃이 사망 상태라면 락온을 해제합니다.

4 플레이어에서 타깃까지의 방향 벡터를 구하고, 방향 벡터를 이용해서 타깃까지의 방위각을 구합니다.

5 플레이어에서 타깃까지의 방향 벡터를 구하고, 방향 벡터를 이용해서 타깃까지의 방위각을 구합니다.

6 카메라가 타깃을 보도록 카메라 행렬을 갱신합니다.

```
IGameObject* CCollider::Target_LockOn(_vector _vPosition, _vector _vLookDirection)
{
    if (m_CollisionObjets.size() == 0)
        return nullptr;

    _float fDistance = { 1000.f };
    IGameObject* pReturnGameObject = { nullptr };
    _vector vLookDirection =
XMVectorSetW(XMVector3Normalize(XMVectorSetY(_vLookDirection, 0.f)), 0.f);

    for (auto pGameObject : m_CollisionObjets)
    {
        _vector vDirection = static_cast<CTransform*>(pGameObject-
>Get_Component(TEXT("Com_Transform")))->Get_State(CTransform::STATE_POSITION) -
_vPosition;
        _vector vDirectionNormal =
XMVectorSetW(XMVector3Normalize(XMVectorSetY(vDirection, 0.f)), 0.f);

        _float fAngle = atan2f(XMVectorGetX(XMVector3Dot(
XMVectorSet(0.f, 1.f, 0.f, 0.f), XMVector3Cross(vLookDirection,
vDirectionNormal)))
            , XMVectorGetX(XMVector3Dot(vLookDirection, vDirectionNormal)));
        if (XMConvertToDegrees(fabsf(fAngle)) <= 15.f)
        {
            _float fLength = XMVectorGetX(XMVector3Length(vDirection));
            if (fLength < fDistance)
            {
                fDistance = fLength;
                pReturnGameObject = pGameObject;
            }
        }
    }
    return pReturnGameObject;
}
```



<계층형 상태 머신 순서도>

계층형 상태 머신

기술 정의

유한 상태 기계 머신에 계층 구조를 도입한 확장 모델

사용 목적

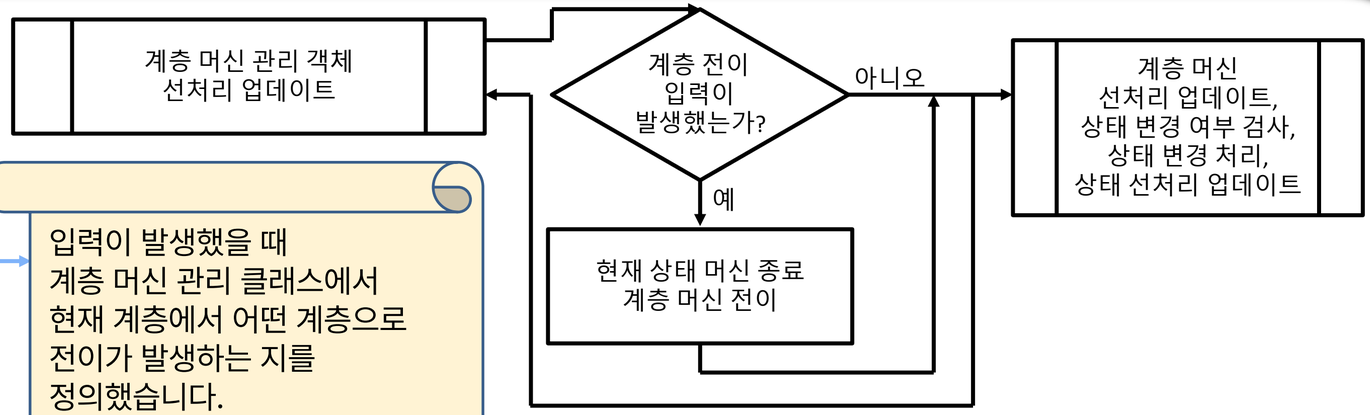
상태를 항목별로 분리해서 전이 관계를 단순화하고 상태 로직의 재사용성을 높이기 위해 사용했습니다.

구현 방법

계층간 상태 전이를 관리하기 위해 계층 머신을 멤버로 소유한 계층 머신 관리 클래스와 상태를 항목별로 나눠 관리하기 위해 실질적인 상태를 멤버로 소유한 계층 머신 클래스, 그리고 실질적인 상태 클래스를 만들어서 사용했습니다.

```
void CPlayer_FSMContainer::Priority_Update(_float _fTimeDelta)
{
    switch (m_iCurMachineIndex)
    {
    case STATE_IDLE:
        if (m_pGameInstance->Get_DIKeyState(DIK_W) & 0x80
            || m_pGameInstance->Get_DIKeyState(DIK_S) & 0x80
            || m_pGameInstance->Get_DIKeyState(DIK_A) & 0x80
            || m_pGameInstance->Get_DIKeyState(DIK_D) & 0x80)
        {
            Change_Machine(STATE_MOVE);
        }
        else if (m_pGameInstance-
>Get_DIMouseInputState(MOUSEKEYSTATE::DIM_LB)
== MOUSEINPUTSTATE::DIMI_LBD
            || m_pGameInstance-
>Get_DIMouseState(MOUSEKEYSTATE::DIM_RB) & 0x80
            || m_pGameInstance-
>Get_DIMouseInputState(MOUSEKEYSTATE::DIM_RB)
== MOUSEINPUTSTATE::DIMI_RBU)
        {
            Change_Machine(STATE_ATTACK);
        }
        else if(m_pGameInstance->Get_DIKeyInputState(DIK_SPACE)
== KEYBOARDINPUTSTATE::KEY_DOWN)
        {
            Change_Machine(STATE_AVOID);
            m_pGameInstance->StopSound(SOUND_EFFECT);
            m_pGameInstance-
>PlayMySound(TEXT("Player_Dodge.wav"), SOUND_EFFECT, 1.f,
false);
        }
        break;
    case STATE_MOVE:
        // 중략
    default:
        m_iCurMachineIndex = m_iCurMachineIndex;
        break;
    }
    __super::Priority_Update(_fTimeDelta);
}
```

<계층 머신 관리 클래스 코드>



<코드 진행 단계>

입력이 발생했을 때
계층 머신 관리 클래스에서
현재 계층에서 어떤 계층으로
전이가 발생하는 지를
정의했습니다.
계층머신 관리 클래스에
계층을 열거형으로 선언해서
가독성을 높였습니다.

```
void CPlayer_FSMContainer::Change_Machine(STATETYPE _eState)
{
    _int iPreAnimIndex = { };
    m_Machines[m_iCurMachineIndex]->Machine_Leave(&iPreAnimIndex);
    m_iCurMachineIndex = _eState;
    if (_eState == STATE_HIT)
        m_Machines[m_iCurMachineIndex]->Set_Direction(m_fAngle);
    m_Machines[m_iCurMachineIndex]->Machine_Enter(&iPreAnimIndex);
}
```

현재 계층 머신의
종료 함수를 호출하고,
변경되는 계층 머신의
시작 함수를 호출합니다.
계층 머신 종료 함수와
시작 함수는 기본적으로
상태 종료와 시작 함수를
호출하지만,
가상화를 했기때문에
세부 사항에 차이를 줄 수
있습니다.

```
void IFSMContainer::Priority_Update(_float _fTimeDelta)
{
    if (m_iNumCategory == 0
        || m_iCurMachineIndex >= m_iNumCategory)
        return;

    m_Machines[m_iCurMachineIndex]->Priority_Update(_fTimeDelta);
}
```

현재 계층 머신의
선처리 업데이트를
호출합니다.

```
void CPlayer_AttackMachine::Priority_Update(_float _fTimeDelta)
{
    switch (m_iCurState)
    {
    case ATK_NORMAL:
        if (m_pGameInstance->Get_DIKeyInputState(DIK_LSHIFT)
== KEYBOARDINPUTSTATE::KEY_PRESS
            && m_pGameInstance->Get_DIMouseInputState(MOUSEKEYSTATE::DIM_RB)
== MOUSEINPUTSTATE::DIMI_RBD
            && m_States[m_iCurState]->Get_Duration() > 1.f)
        {
            _int iPreAnimIndex = { };
            m_States[m_iCurState]->State_Leave(&iPreAnimIndex);
            m_iCurState = ATK_UPPER;
            m_States[m_iCurState]->State_Enter(&iPreAnimIndex);
        }
        break;
    case ATK_UPPER:
        // 중략
    default:
        break;
    }

    __super::Priority_Update(_fTimeDelta);
}
```

<계층 머신 클래스 코드>

```
void CPlayer_Attack_Normal::Priority_Update(_float _fTimeDelta)
{
    switch (m_iCurAnimIndex)
    {
    case ATTACK_WEAK1:
        if ((m_pGameInstance->Get_DIMouseInputState(MOUSEKEYSTATE::DIM_LB)
== MOUSEINPUTSTATE::DIMI_LBD)
            && (m_fDuration > 1.f))
        {
            m_iPreAnimIndex = m_iCurAnimIndex;
            m_iCurAnimIndex = ATTACK_WEAK2;
            m_fDuration = 0.f;
        }
        break;

    case ATTACK_WEAK2:
        // 중략
    default:
        break;
    }

    __super::Priority_Update(_fTimeDelta);
}
```

입력이 발생했을 때 다음에 진행할 애니메이션을 정의했습니다.

<상태 클래스 코드>

입력이 발생했을 때 계층 머신 클래스에서
현재 상태에서 어떤 상태로 전이가 발생하는 지 정의했습니다.
계층 머신 클래스에 상태를 열거형으로 선언해서
가독성을 높였습니다.

```
void CPlayer_Attack_Normal::State_Leave(_int* _pPreAnimIndex)
{
    m_iCurAnimIndex = ANIM_END;
    *_pPreAnimIndex = m_iCurAnimIndex;
    __super::State_Leave(_pPreAnimIndex);
}
```

```
void IPlayer_AttackState::State_Leave(_int* _pPreAnimIndex)
{
    m_bAnimLoop = { false };
    m_bAnimFinished = { false };
    m_fDuration = 0.f;
    m_iPreAnimIndex = -1;
}
```

```
void CPlayer_Attack_Normal::State_Enter(_int* _pPreAnimIndex)
{
    m_iCurAnimIndex = ATTACK_WEAK1;
    __super::State_Enter(_pPreAnimIndex);
}
```

```
void IPlayer_AttackState::State_Enter(_int* _pPreAnimIndex)
{
    m_iPreAnimIndex = *_pPreAnimIndex;
    m_bAnimLoop = false;
}
```

```
void IStateMachine::Priority_Update(_float _fTimeDelta)
{
    m_States[m_iCurState]->Priority_Update(_fTimeDelta);
}
```

계층 머신
선처리 업데이트,
상태 변경 여부 검사,
상태 변경 처리,
상태 선처리 업데이트

상태 업데이트,
애니메이션 변경 여부 검사,
애니메이션 변경 처리

<코드 진행 단계>

상태 클래스에
애니메이션의 이름을
열거형으로 선언하고
애니메이션의 인덱스를
열거형의 값으로 선언해서
가독성과 편의성을 높였습니다.
상태를 종료하면서 상태에
저장된 값을 초기값으로
대입했습니다.

상태 진입 시작 애니메이션을 대입하고
보간을 위해 이전 애니메이션의
인덱스를 대입합니다.

현재 상태의
선처리 업데이트를 호출합니다.

<디졸브예시>

```
    if (m_eState == STATE_DEATH)
        m_fDissolveTime += _fTimeDelta * 0.15f;
```

객체 후처리 업데이트에서
객체가 사망상태일 때, 디졸브 값을 증가시킵니다.

```
    vector vNoise = g_DissolveTexture.Sample(LinearClampSampler,
In.vTexcoord);

    if (vNoise.r < g_fDissolveTime)
        discard;
```

픽셀 셰이더 단계에서 샘플링한 노이즈 텍스쳐의 r 값이 디졸브 값보다 작다면 픽셀을 버립니다.

디졸브

기술 정의

서서히 사라지거나 서서히 나타나는 효과

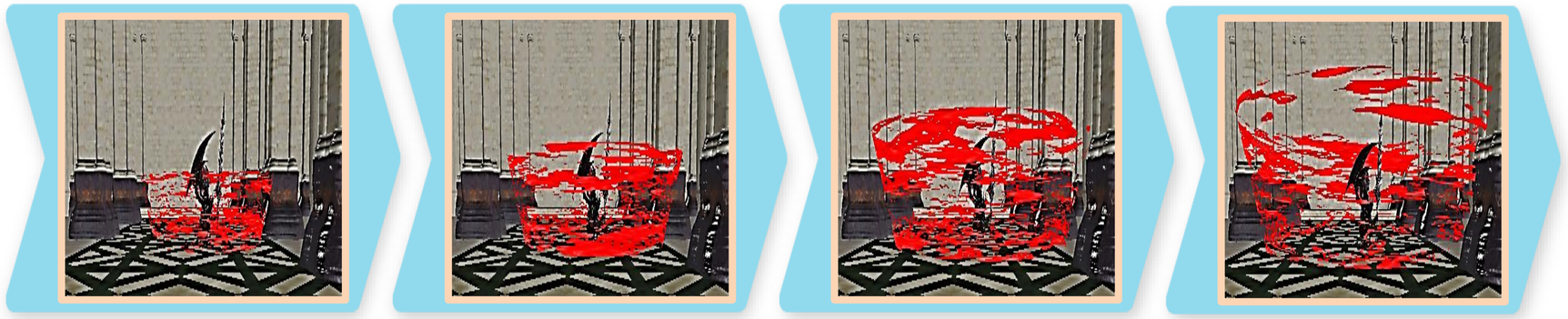
사용 목적

객체가 사망상태에 진입하고 일정 시간이 지난 후 사라지는데 한 번에 사라지니 어색함이 있어 자연스러움을 위해 사용했습니다.

구현 방법

사망상태에 진입 시 디졸브 값을 증가시키고, 노이즈 텍스쳐를 활용해 샘플링된 노이즈 텍스쳐의 r 값이 디졸브 값보다 작으면 픽셀 셰이더 단계에서 픽셀을 버렸습니다.

<스프라이트 파티클 이펙트 예시>



<메시 스프라이트 파티클 이펙트 예시1>

<메시 스프라이트 파티클 이펙트 예시2>

스프라이트 이펙트

기술 정의

2D 스프라이트로 여러 가지 모양이 있는 물체를 구현하는 기술

사용 목적

이펙트가 변화하는 듯한 효과를 주기위해 사용했습니다.

구현 방법

텍스쳐의 UV 좌표를 재맵핑해서 구현했습니다.

```
void CSpriteObject::Update(_float _fTimeDelta)
{
    if (m_bActive == false)
        return;

    XMStoreFloat4x4(&m_CombinedWorldMatrix
        , XMLoadFloat4x4(m_pTransformCom->Get_WorldMatrix_Ptr())
        * XMLoadFloat4x4(m_pOwnerMatrix));

    m_fTrackPosition += m_iNumTexture * _fTimeDelta * m_fTimeMagnification;
    if(m_iNumTexture != 0)
    {
        if (m_fTrackPosition >= (m_vUVSize.x))
        {
            m_fTrackPosition = fmodf(m_fTrackPosition, (m_vUVSize.x));
            if (++m_vUVIndexed.x >= m_vUVMaxed.x)
            {
                m_vUVIndexed.x = 0.f;
                if (++m_vUVIndexed.y >= m_vUVMaxed.y)
                    m_vUVIndexed.y = 0.f;
            }
            _float2 vSize = m_vUVSize;
            vSize.y = vSize.x;
            m_pVIBufferCom->Change_AtlasIndex(m_vUVIndexed, vSize);
        }
    }

    __super::Compute_Depth();
}
```

이펙트가 비활성 상태라면 업데이트 하지 않습니다.

이펙트의 진행 방향이나 회전 등을 고려하기 위해
이펙트 생성 시 정의된 부모 행렬을 곱합니다.

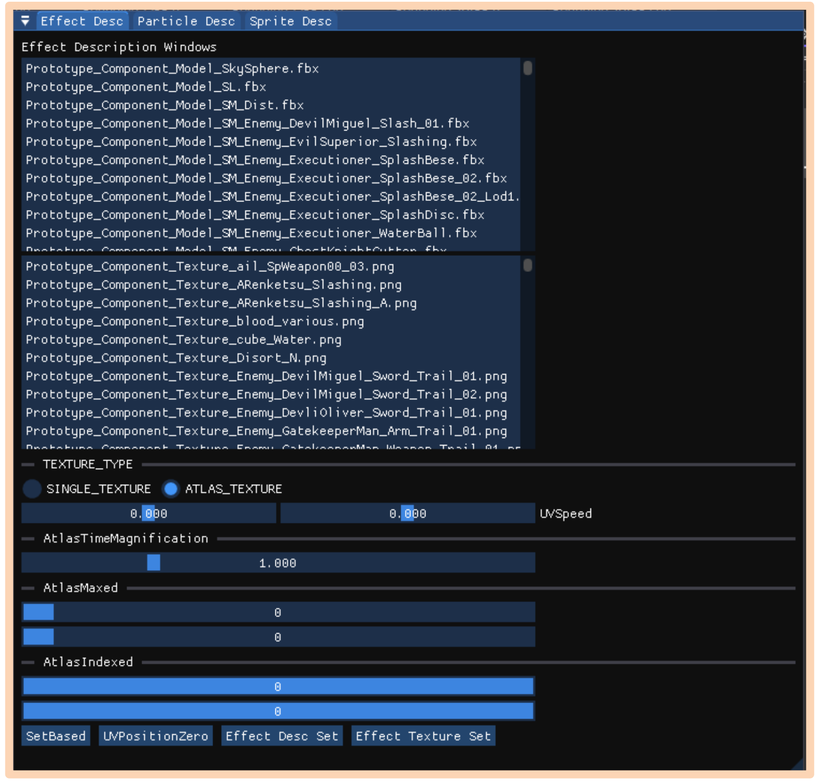
이펙트의 스프라이트 텍스쳐 중 현재 텍스쳐의 생명
주기를 연산합니다.

일정 시간마다 그려지는 텍스쳐가 바뀌도록 연산해서
생명 주기가 지나면 다음 텍스쳐를 출력하기 위해
UV 시작 좌표를 변경합니다.

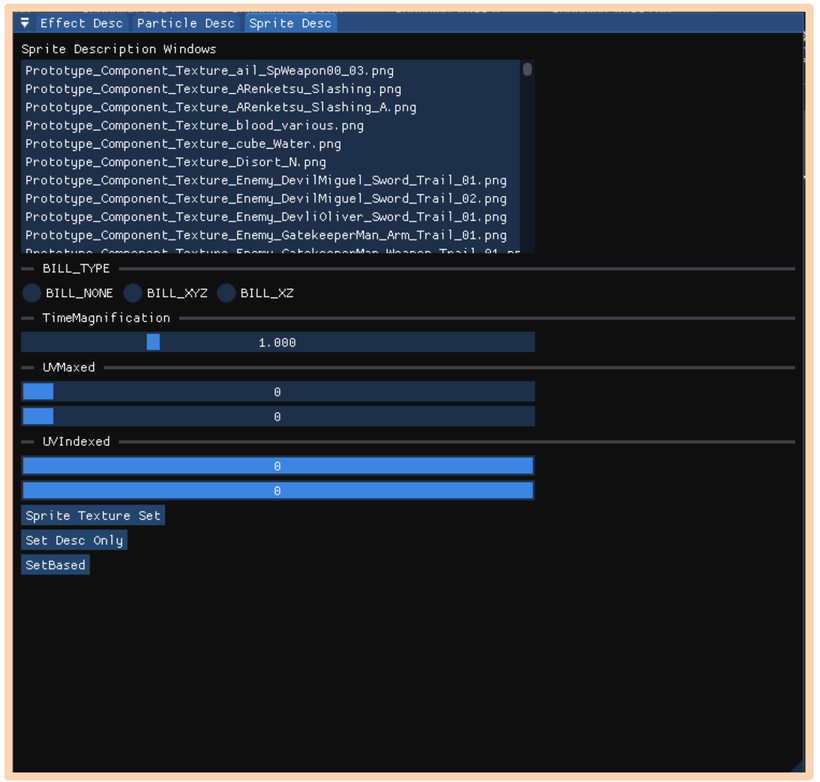
```
void CVIBuffer_Rect::Change_AtlasIndex(_float2 _vUVIndex, _float2
_vUVSize)
{
    D3D11_MAPPED_SUBRESOURCE VerticesSubResources = { };

    m_pContext->Map(m_pVB, 0, D3D11_MAP_WRITE_NO_OVERWRITE, 0,
&VerticesSubResources);

    VTXPOSTEX* pVertices = static_cast<VTXPOSTEX*>
(VerticesSubResources.pData);

    pVertices[0].vTexcoord = _float2(_vUVIndex.x * _vUVSize.x,
_vUVIndex.y * _vUVSize.y);
    pVertices[1].vTexcoord = _float2(pVertices[0].vTexcoord.x +
_vUVSize.x, pVertices[0].vTexcoord.y);
    pVertices[2].vTexcoord = _float2(pVertices[0].vTexcoord.x +
_vUVSize.x, pVertices[0].vTexcoord.y + _vUVSize.y);
    pVertices[3].vTexcoord = _float2(pVertices[0].vTexcoord.x,
pVertices[0].vTexcoord.y + _vUVSize.y);

    m_pContext->Unmap(m_pVB, 0);
}
```

텍스쳐의 UV 시작 좌표인
좌상단 점에서 우상단, 우하단,
좌하단의 점들의 좌표를 연산해서
UV 좌표를 재맵핑합니다.

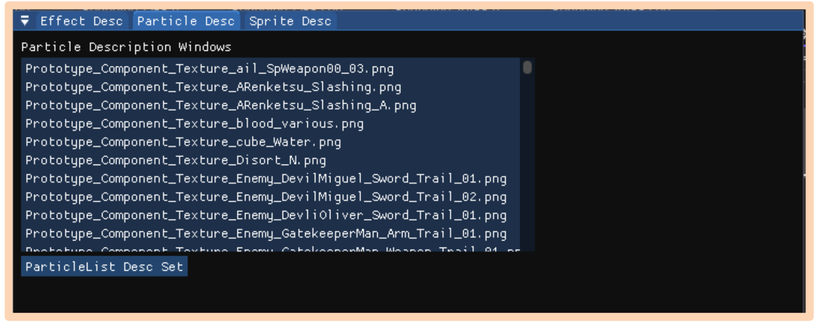
이펙트 툴



<메시 이펙트 툴 예시>



<스프라이트 이펙트 툴 예시>



<파티클 이펙트 툴 예시>

기술 정의

이펙트를 만들기 위한 툴 구현

사용 목적

출력되는 이펙트가 어떻게 그려지는지 확인하기 위해 사용했습니다.

구현 방법

출력하는 이펙트의 UV 좌표나 출력 범위, 텍스쳐를 변경해서 이펙트가 어떻게 그려지는지 확인할 수 있게 했습니다.

DirectX11을 이용한 3D 게임 모작

원작게임

바이오하자드 RE:4

작업기간

2025.04.28 ~ 2025.06.30

작업인원

5인 팀 게임 모작

담당 업무

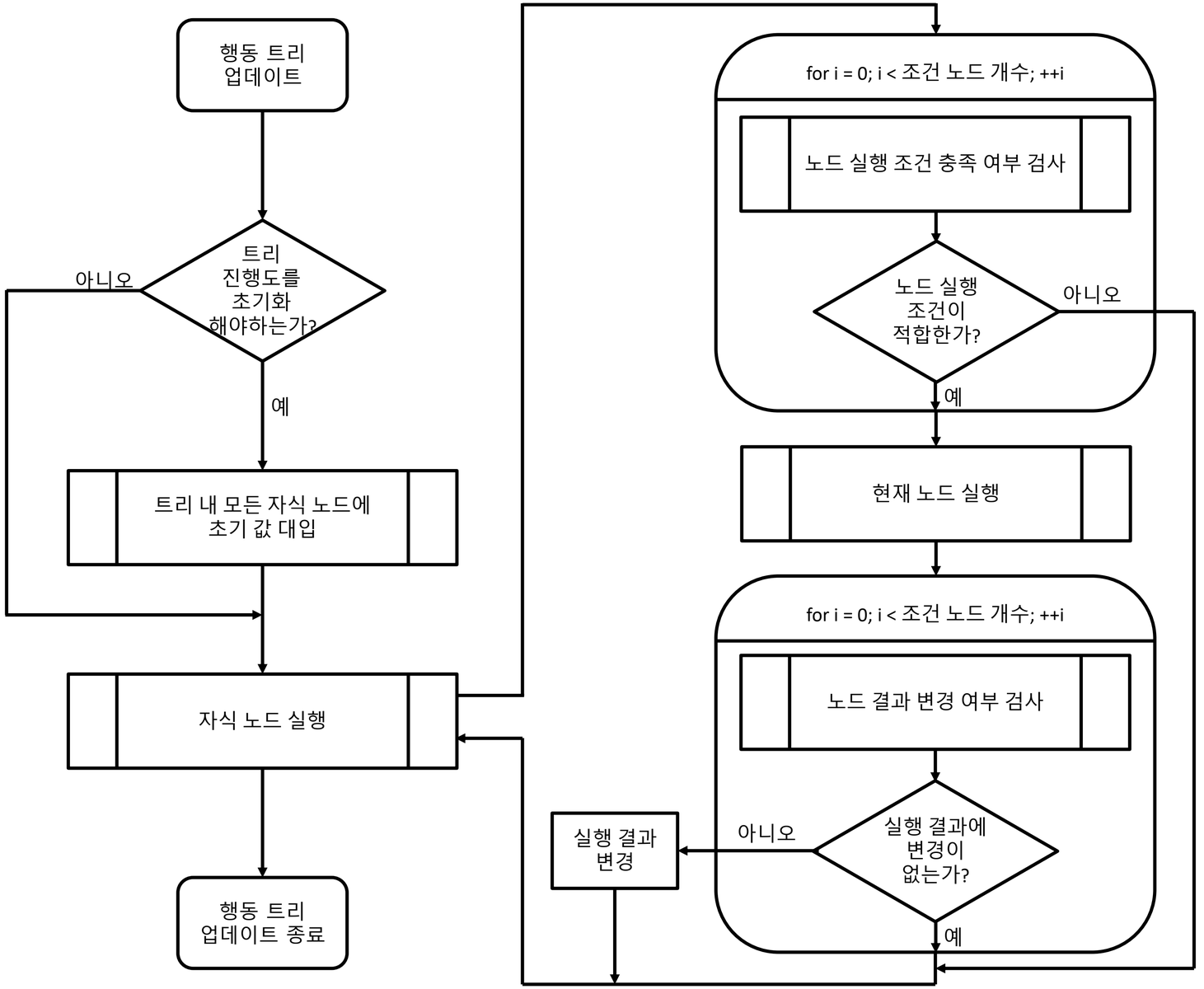
몬스터 담당

사용 언어

C++
HLSL 5.0

제작 환경

Visual Studio 2022
IMGUI
Assimp
PhysX



<행동 트리 순서도>

행동 트리

기술 정의

행동을 트리 구조로 만들어 행동을 제어하는 모델

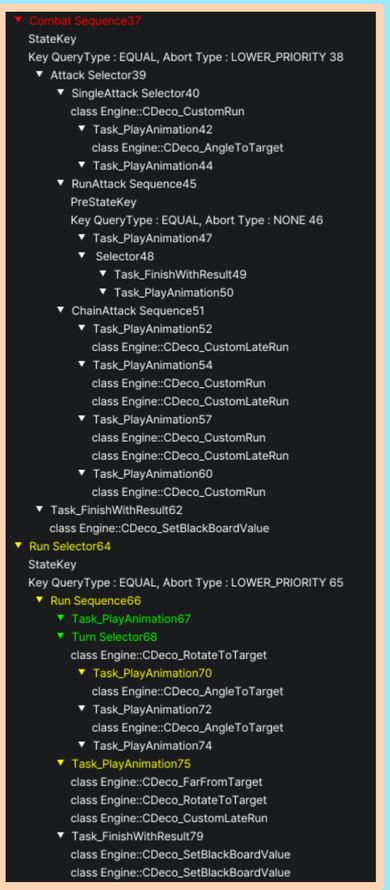
사용 목적

행동을 모듈화해서 구조적으로 관리하고, 여러 객체에서 공통 노드를 재사용하기 위해 행동 트리를 사용했습니다.

구현 방법

노드를 조건 노드와 제어 노드, 그리고 실행 노드로 나눠 구현했습니다.
노드를 가진 트리를 만들고, 트리가 노드를 실행하면 노드는 조건 노드를 통해 실행 여부를 판단하고, 제어 노드를 통해 자식 노드를 탐색하며, 실행 노드를 통해 행동을 실행하도록 구현했습니다.

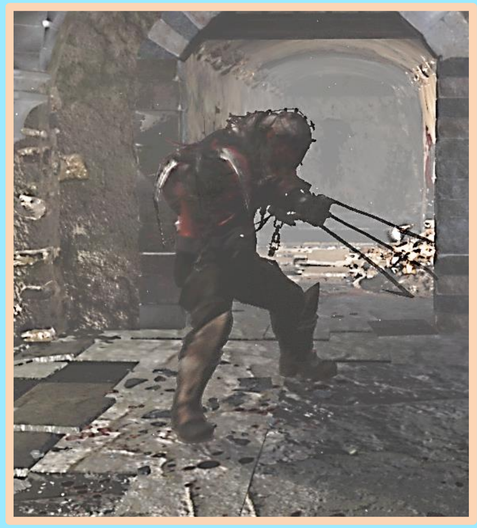
행동 트리를 이용한 행동 제어 예시



<행동 트리 에디터 툴 예시>

<행동 제어 - 플레이어 감지 예시>

<행동 제어 - 플레이어 추적 예시>



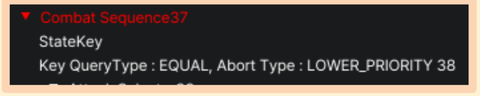
<행동 제어 – 공격 예시>

행동 순서	감지 행동	추적 행동	추적 행동	공격 행동
노드 이름	Task_PlayAnimation67	Task_PlayAnimation70	Task_PlayAnimation75	Task_PlayAnimation47
조건 노드	없음	플레이어가 정면에 위치해야한다.	없음	직전 행동이 추적 행동이어야 한다.
결과 변경 노드	없음	없음	플레이어까지의 거리나 방향에 따라 행동을 조기 종료할 수 있다.	없음
상위 제어 노드	하위 노드를 모두 실행한다.	하위 노드 중 하나만 실행한다.	하위 노드를 모두 실행한다.	하위 노드를 모두 실행한다.

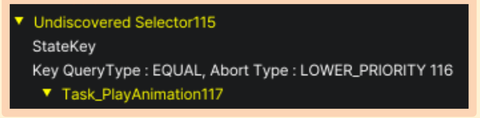
<행동 진행 순서표 예시>

```
CBT_CompositeNode::TYPE_RESULT CBT_CompositeNode::Receive_Excute(CGameObject* pOwnerObject)
{
    if (COMPOSITE_PARALLEL != m_eCompositeType
        ? (CBT_BaseNode::RESULT_END == m_eResultType)
        : (CBT_BaseNode::RESULT_END == m_ResultTypes[m_iExecuteIndex]))
    {
        for (auto& pDecorator : m_MyDecoratorNodes)
        {
            if (false == pDecorator->Run(pOwnerObject))
            {
                if ((COMPOSITE_PARALLEL != m_eCompositeType)
                    ? ((m_eResultType = CBT_BaseNode::RESULT_FAIL) == m_eResultType)
                    : ((m_ResultTypes[m_iExecuteIndex] = CBT_BaseNode::RESULT_FAIL) !=
m_ResultTypes[m_iExecuteIndex]))
                    return m_eResultType;
            }
        }
    }

    else if ((COMPOSITE_PARALLEL != m_eCompositeType)
        ? (CBT_BaseNode::RESULT_FAIL == m_eResultType || CBT_BaseNode::RESULT_ABORT == m_eResultType)
        : (CBT_BaseNode::RESULT_FAIL == m_ResultTypes[m_iExecuteIndex] || CBT_BaseNode::RESULT_ABORT
== m_ResultTypes[m_iExecuteIndex]))
    {
        m_ChildNodes[m_iExecuteIndex]->Set_ResultType(CBT_BaseNode::RESULT_FAIL);

        if (COMPOSITE_PARALLEL != m_eCompositeType)
            return m_eResultType;
    }

    m_eResultType = Run(pOwnerObject);

    if (CBT_BaseNode::RESULT_FAIL == m_eResultType)
        return CBT_BaseNode::RESULT_FAIL;

    for (auto& pDecorator : m_MyDecoratorNodes)
    {
        CBT_BaseNode::TYPE_RESULT eType = pDecorator->Late_Run(pOwnerObject);
        if (CBT_BaseNode::RESULT_SUCCESS == eType)
            return m_eResultType = CBT_BaseNode::RESULT_SUCCESS;
        else if (CBT_BaseNode::RESULT_ABORT == eType)
            return m_eResultType = CBT_BaseNode::RESULT_ABORT;
    }

    return m_eResultType;
}
```

현재 노드에 진행 조건이
충족됐는지 확인합니다.

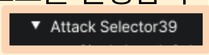


<조건 노드 미달 예시>

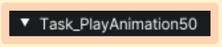


<조건 노드 충족 예시>

현재 노드를 실행합니다.



<제어 노드 예시>



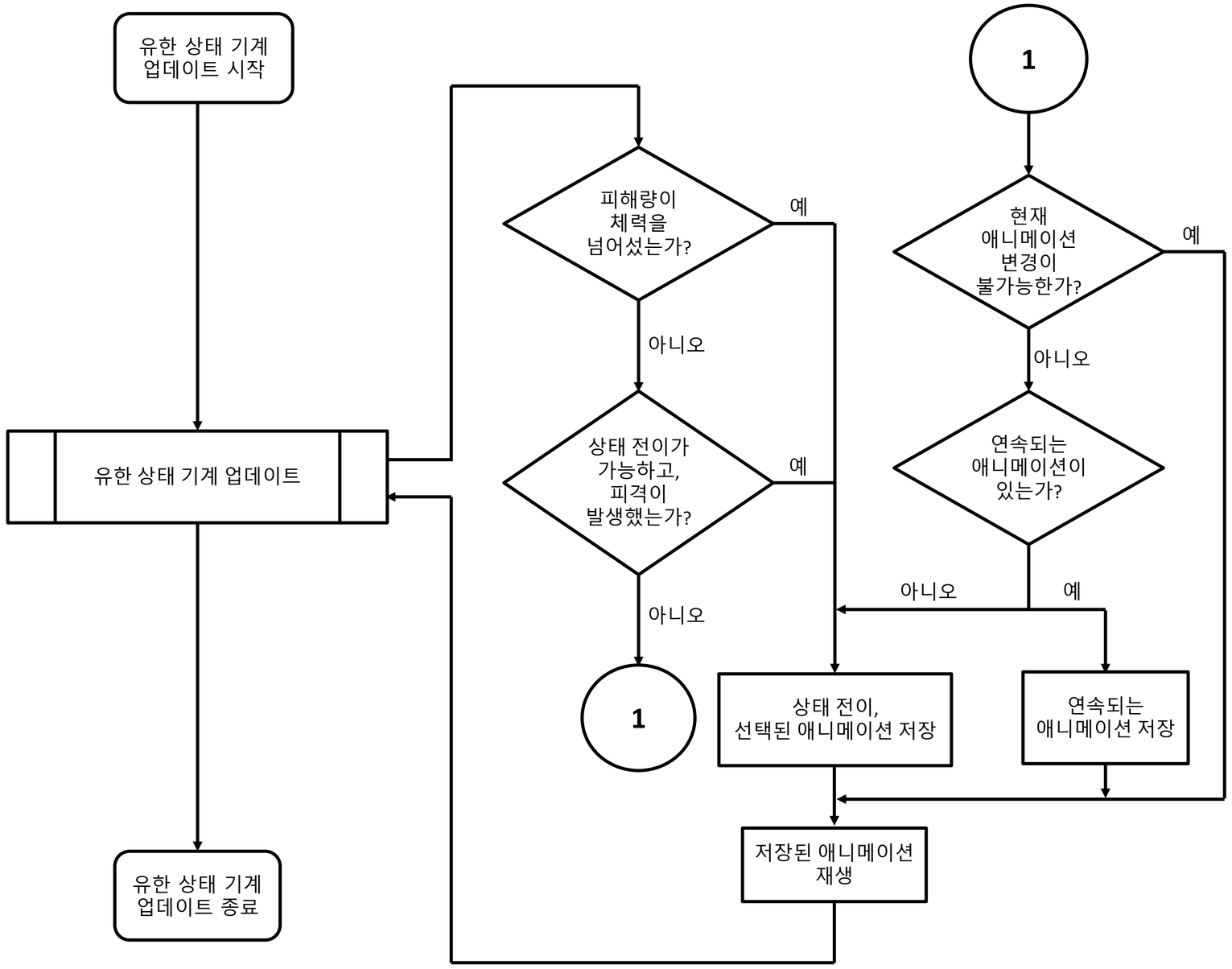
<실행 노드 예시>

현재 노드의 진행 결과가
변경될 필요가 있는지
확인하고, 조건을 충족한다면
진행 결과를 변경합니다.

< 행동 제어 – 대기 예시 >

< 행동 제어 – 방향 전환 예시 >

< 행동 제어 - 이동 예시 >



<유한 상태 기계 순서도>

유한 상태 기계

기술 정의

상태를 기반으로 행동을 제어하는 모델

사용 목적

행동 트리의 노드 설계에 시간이 많이 소요되어
구현 속도를 높이기 위해 유한 상태 기계
로 전환했습니다.

구현 방법

열거형으로 상태를 정의하고 switch문으로 현재
상태를 진행했습니다.
상태 전이가 발생하면 현재 상태를 종료하고
새로운 상태에 진입하도록 구현했습니다.

```
HRESULT CEasyFSM_Soldier::Update(_float fTimeDelta)
{
    for (auto& Pair : m_AnimParts)
    {
        if (Pair.second == false)
            continue;

        _string strCurAnimName = m_strPlayAnimName;

        m_fTimeAccu += fTimeDelta;

        State_Update(Pair.first, fTimeDelta);

        if (strCurAnimName != m_strPlayAnimName)
            m_pOwnerCompositeArmature
->Select_Animation(Pair.first, m_strPlayAnimName,
m_bLoop);
    }

    return S_OK;
}
```

애니메이션 변경 여부를
확인합니다.
애니메이션을 담아둔
컨테이너를 순회하기 때문에
애니메이션이 변경된
경우에만 수행합니다.

< 걷기 상태 – 이동 시작 >

< 걷기 상태 – 이동 중 >

< 걷기 상태 – 이동 종료 >

```
_bool CEasyFSM_Soldier::State_Change(_string strPart
, CHARACTERSTATE eState)
{
    if (eState == *m_pState)
        return false;

    State_Exit(strPart, *m_pState);
    m_ePreState = *m_pState;

    m_pVariableContext->Set_Variable<_int>
("StateKey", (_int)eState);

    State_Enter(strPart, *m_pState);

    return true;
}
```

현재 상태를 종료하기 전에
정리가 필요한 데이터가
있다면 해당 데이터를
정리하고 상태를 변경합니다.

```
void CEasyFSM_Soldier::State_Enter(_string strPart, CHARACTERSTATE eState)
{
    switch (*m_pState)
    {
        // 중략
    case CHARST_WALK:
        m_AnimationLink.push_back({ m_ReplaceAnimNames["Walk_Continue_Start_AL"], false });
        m_AnimationLink.push_back({ m_ReplaceAnimNames["Walk_Continue_Loop_AA"],true });
        m_AnimationLink.push_back({ m_ReplaceAnimNames["Walk_Continue_End_AA"],false });
        Set_NextAnimQueue();

        m_fWaitTime = 6.f;
        m_fTimeAccu = 0.f;
        break;
        // 중략
    }
}
```

변경된 상태로 진입해서 진행할
애니메이션을 저장합니다.
연속된 애니메이션일 경우
컨테이너에 모두 쌓고
애니메이션 변경이 필요할 때
먼저 저장한 순서대로 꺼냅니다.

```
void CEasyFSM_Soldier::State_Update(_string strPart, _float fTimeDelta)
{
    if (*m_pHP <= *m_pDamage
&& (0 == ((CSB_DEAD | CSB_LATE_DEAD) & m_iFlagActive)))
    {
        if (true == State_Change(strPart, CHARST_DEAD))
            return;
    }
  // 중략

    switch (*m_pState)
    {
        // 중략
    case CHARST_WALK:
        if (true == m_pOwnerCompositeArmature->Get_Animator(strPart)->Get_IsLoop()
|| true == m_pOwnerCompositeArmature->Get_AnimEndNotify(strPart))
        {
            Calc_FromOwnerToTarget();

            m_pOwner->Get_TransformCom()
->Rotation_Dir_Smooth(XMLoadFloat4(&m_vToTargetDirNor), fTimeDelta);

            if (m_fDistance <= (m_fAtkRange - 2.f) && m_fFrontAngle >= 0.f
&& fabsf(m_fAngle) <= XMConvertToRadians(10.f))
            {
                State_Change(strPart, CHARST_ATTACK);
                return;
            }

            if (true == m_AnimationLink.empty())
            {
                State_Change(strPart, CHARST_IDLE);
                return;
            }
            else
            {
                Set_NextAnimQueue();
                return;
            }

        }
        else if (false == m_AnimationLink.empty())
        {
            Calc_FromOwnerToTarget();
            m_pOwner->Get_TransformCom()
->Rotation_Dir_Smooth(XMLoadFloat4(&m_vToTargetDirNor), fTimeDelta);
        }
        break;
    }
}
```

대부분의 상태에서 전이가능한
우선도가 높은 상태로 전이가
필요한지 확인합니다.

조건에 부합한 경우
해당 상태로 전이합니다.

컨테이너에 먼저 저장된
애니메이션을 빼서 진행중인
애니메이션으로 저장합니다.

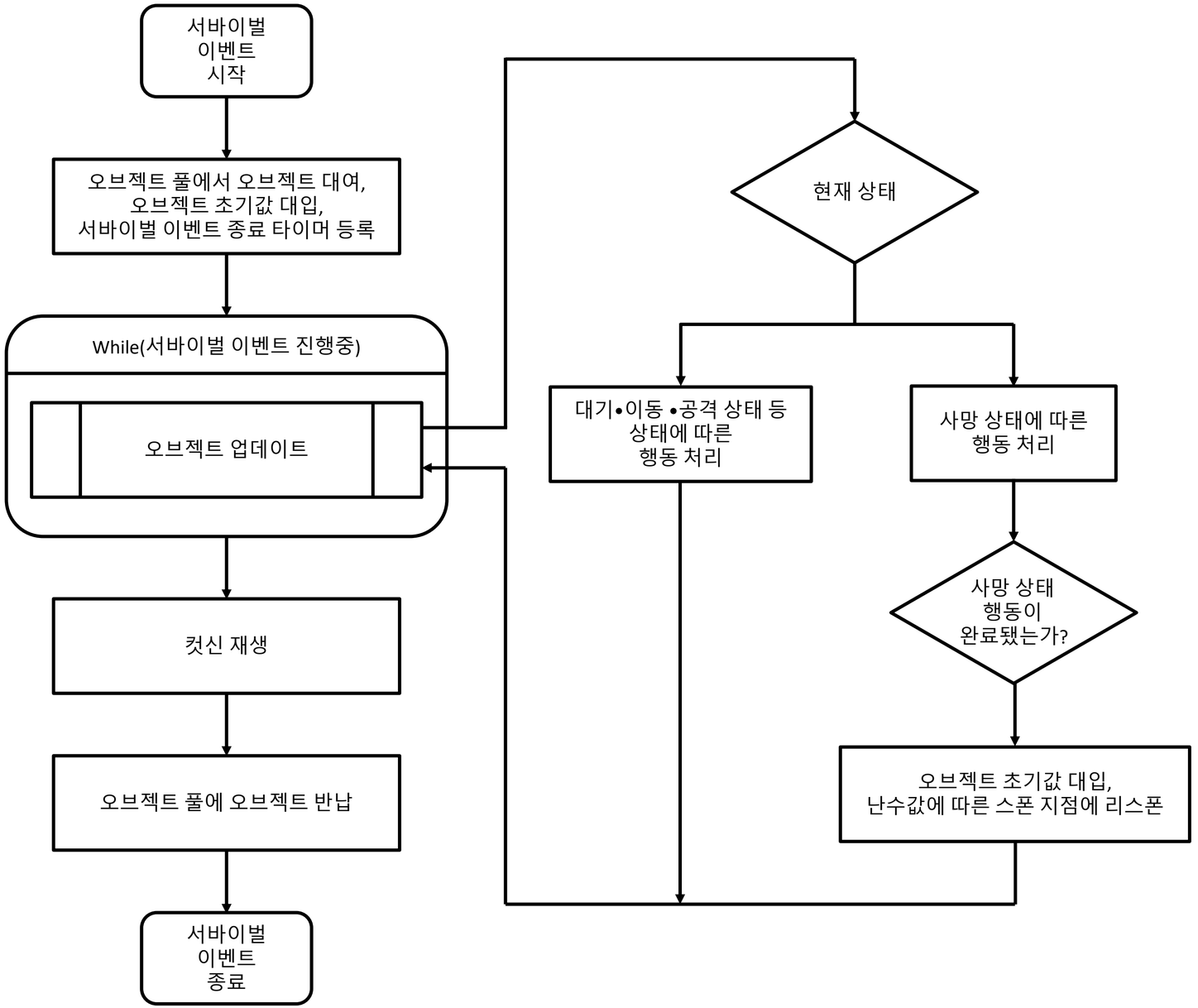
몬스터에서 플레이어까지의
거리와 각도를 구하고,
해당 방향으로 조금씩 회전한다.

< 공격 상태 예시 >

< 그로기 상태 예시 >

< 사망 상태 예시 >

< 피격 상태 예시 >



<서바이벌 이벤트 순서도>

서바이벌 이벤트

기술 정의

정해진 시간동안 몬스터들로부터 살아남는 이벤트

사용 목적

일상 전투와 보스 전투 외 컨텐츠를 위해 만들었습니다.

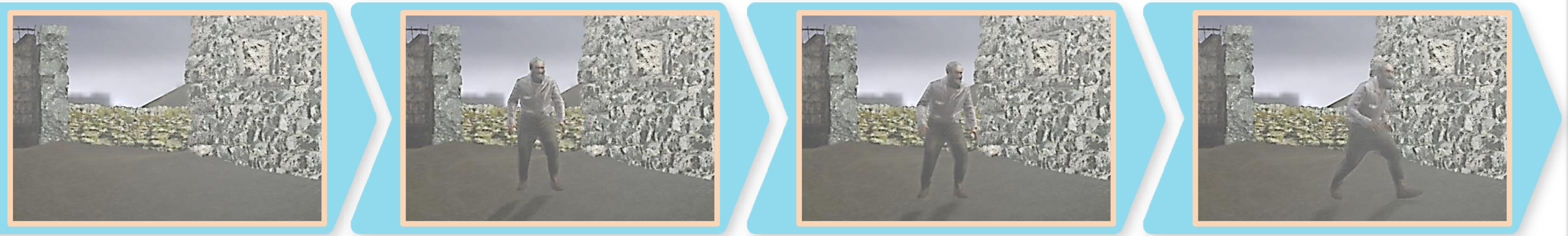
구현 방법

몬스터 풀링을 통해 몬스터들을 미리 만들어두고, 몬스터가 사망처리가 되면 초기값을 대입하고 정해진 스폰장소들 중 랜덤한 위치에 리스폰했습니다.

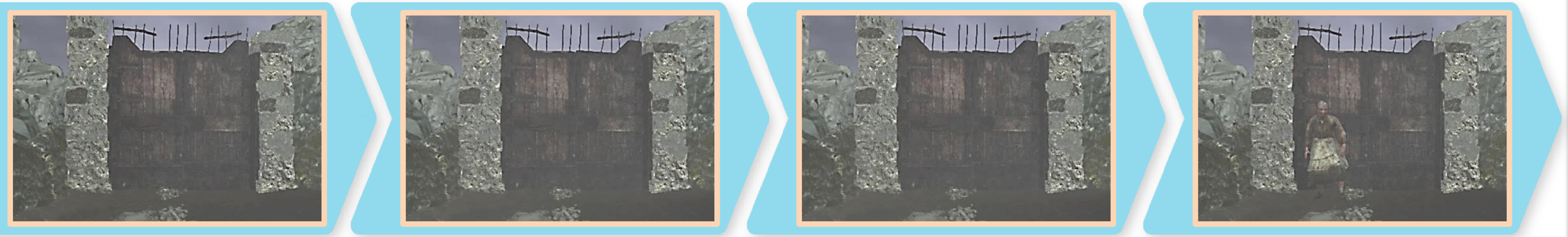
서바이벌 이벤트 시작 후 시간별 각 스폰 지점

스폰 지점 A

스폰 지점 B



스폰 지점 C



0 초 — 2 초 — 4 초 — 6 초

```
pActiveTrigger->Get_Bound()->Add_CollisionCallBack([&, pActiveTrigger](CBound* pBound,
COLLISION_INFO* pInfo)
    {
      // 중략

        for (auto& CommonMonster : pActiveTrigger->Get_AttachedObjects())
        {
            CMonsterCommonObject* pMonster = dynamic_cast<CMonsterCommonObject*>
(CommonMonster);
            if (nullptr == pMonster)
                continue;

            _float3 vPos = {};
            _float3 vRot = { 0.f, 0.f, 0.f };
            switch (CMyUtils::Get_RandomInt(0, 100) % 3)
            {
            case 0:
                vPos = g_vRespawnPointA;
                break;
            case 1:
                vPos = g_vRespawnPointB;
                break;
            case 2:
                vPos = g_vRespawnPointC;
                break;
            }
            pMonster->Set_ObjectMatrixQueue(vPos, vRot);
            pMonster->Set_Target();
            pMonster->IsTypeForCutscene();

            m_pGameInstance->Add_Timer("SpawnTimer", fTimer, [&, CommonMonster]() {
                CommonMonster->Initialize_PoolingObject(nullptr);
                CommonMonster->Set_Sleep(false);
              // 중략

                });
            fTimer += 2.f;
        }

      // 중략

        m_pGameInstance->Add_Timer("BloodEventEnd", g_fBloodEventTimer, [&]() {
            g_bBloodEvent = false;

          // 중략

            for (auto& CommonMonster : pActiveTrigger->Get_AttachedObjects())
            {
                CMonsterCommonObject* pMonster = dynamic_cast<CMonsterCommonObject*>
(CommonMonster);
                if (nullptr == pMonster)
                    continue;
                pMonster->End_BloodEvent();
            }

            m_pGameInstance->Add_Timer(CMyUtils::Convert_String(m_szGameObjectTag), 0.f,
[&]() {m_pGameInstance->Remove_Object(pActiveTrigger); }, false);
            g_pVariableContext->Set_Variable<CGameObject*>("ActiveTriggerKey", nullptr);
            });
    });
```

오브젝트 풀에서
오브젝트를
대여하고 초기값을
대입합니다.

서바이벌 이벤트 종료
타이머를 등록하고, 종료될
때 수행해야할 동작들을
람다식을 사용해서
정의합니다.

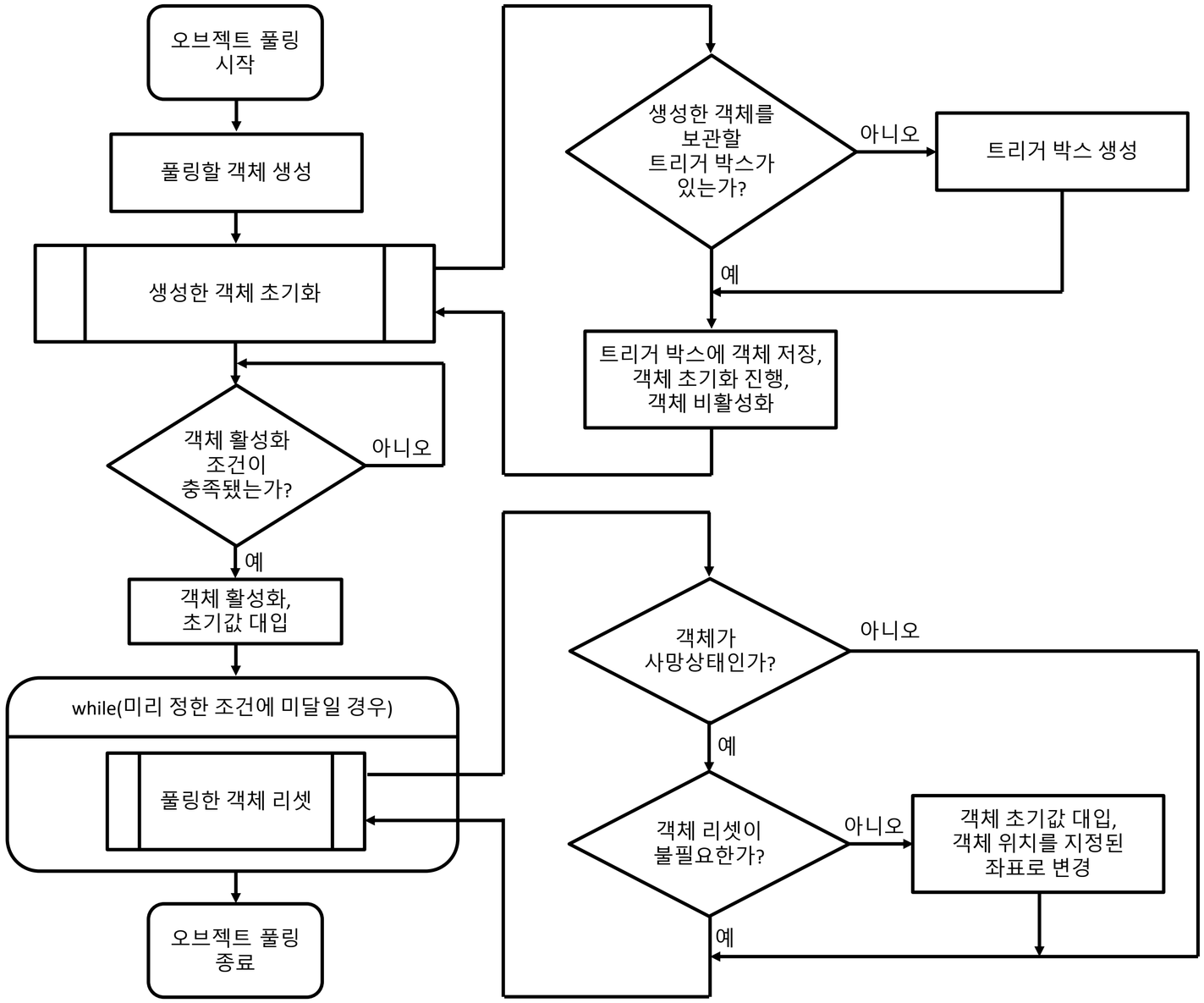
<서바이벌 이벤트 시작 트리거 예시>

<서바이벌 이벤트 시작 최초 스폰 예시>

<서바이벌 이벤트 진행 중 사망 상태 예시>

<서바이벌 이벤트 진행 중 리스폰 예시>

<서바이벌 이벤트 종료 컷신 예시>



<오브젝트 풀링 순서도>

오브젝트 풀링

기술 정의

사용할 객체를 미리 만들어서 풀에 저장해두고 필요한 경우 풀에서 객체를 대여해서 사용하는 기술

사용 목적

로딩할 때 객체를 미리 만들고, 해당 지역에 진입하기 전까지 객체를 비활성화하기 위해 사용했습니다.

구현 방법

객체 초기화 시 전역으로 사용할 트리거 박스가 없다면 트리거 박스를 만들고 박스에 초기화 중인 객체를 담은 후 객체를 비활성화합니다.
플레이어가 트리거 박스에 충돌하면 객체들을 활성화하고, 정해진 횟수동안 객체가 사망상태가 되면 초기 값을 대입하고 지정된 다음 좌표로 객체의 좌표로 변경합니다.

```
HRESULT CMonsterCommonObject::Initialize(void* pArg)
{
  // 중략
        if (nullptr == g_pVariableContext)
        {
                CContext* pContext = dynamic_cast<CContext*>(m_pGameInstance->Clone_Prototype(PROTOTYPE::TYPE_COMPONENT
, LEVEL_STATIC, KEY_PROTO_COM_CONTEXT));
                if (nullptr == pContext)
                    return E_FAIL;

                g_pVariableContext = pContext;

                CTrigger::Trigger_DESC ActiveTriggerDesc = {};
              // 중략

                CTrigger* pActiveTrigger = dynamic_cast<CTrigger*>(m_pGameInstance
->Clone_Prototype(PROTOTYPE::TYPE_GAMEOBJECT, LEVEL_STATIC, TEXT("Prototype_GameObject_Trigger"), &ActiveTriggerDesc));

                pActiveTrigger->Get_Bound()->Add_CollisionCallBack(
[&, pActiveTrigger](CBound* pBound, COLLISION_INFO* pInfo)
                    {
                      // 중략
                        for (auto& CommonMonster : pActiveTrigger->Get_AttachedObjects())
                        {
                            CMonsterCommonObject* pMonster = dynamic_cast<CMonsterCommonObject*>(CommonMonster);
                            if (nullptr == pMonster)
                                continue;

                            _float3 vPos = {};
                            _float3 vRot = { 0.f, 0.f, 0.f };
                          // 중략
                            pMonster->Set_ObjectMatrixQueue(vPos, vRot);

                            m_pGameInstance->Add_Timer("SpawnTimer", fTimer, [&, CommonMonster]() {
                                CommonMonster->Initialize_PoolingObject(nullptr);
                                CommonMonster->Set_Sleep(false);
                                if (CRigidBody* pRigid = CommonMonster->Get_RigidBody())
                                    m_pGameInstance->Attach_Rigid(pRigid);
                                });
                        }
                      // 중략
                    });
                pActiveTrigger->Attach_Object(this);
                g_pVariableContext->Set_Variable("ActiveTriggerKey", pActiveTrigger);
        }
          else
        {
            Safe_AddRef(g_pVariableContext);
            g_pVariableContext->Get_Variable<CGameObject*>("ActiveTriggerKey")->Attach_Object(this);
        }
  // 중략
}
```

<객체 비활성화 예시 1>

<객체 활성화 예시1>

객체를 보관할 풀을
생성합니다.

객체에 초기값 대입과
객체 활성화를 예약합니다.

풀 객체를 저장하고,
풀에 객체를 추가합니다.

<객체 비활성화 예시 2>

<객체 활성화 예시 2>

<보스 몬스터 이미지 1>

<보스 몬스터 이미지 2>

<보스 몬스터 이미지 3>

<보스 몬스터 이미지 4>

보스 몬스터 구현

기술 정의

보스 몬스터 클래스 구현

사용 목적

보스 몬스터만의 기믹을 사용하기 위해서 보스 몬스터 클래스를 개별로 사용했습니다.

구현 방법

기본적인 동작은 FSM에 정의하고 보스 몬스터의 기믹 조건을 보스 몬스터 클래스에 정의했습니다.
컷신을 위한 파이널 페이즈를 별개의 클래스로 분리해서 보스 몬스터가 사망 처리가 끝난 후 파이널 페이즈를 활성화합니다.
이벤트 델리게이트 기반으로 이벤트를 처리합니다.

```
HRESULT CSaddlerObject::Initialize(void* pArg)
{
    MONSTERSADDLEROBJECT_DESC* pDesc
= reinterpret_cast<MONSTERSADDLEROBJECT_DESC*>(pArg);
    // 중략

     if (FAILED(__super::Initialize(pDesc)))
        return E_FAIL;

    if (FAILED(Ready_Components()))
        return E_FAIL;

    if (FAILED(Ready_TriggerHands()))
        return E_FAIL;
    // 중략

    m_pVariableContext->Add_Variable<_int>("StateKey"
, (_int)(CHARST_IDLE));
    // 중략

    if (FAILED(Bind_Delegate()))
        return E_FAIL;

    if (FAILED(Ready_FSM()))
        return E_FAIL;

    if (FAILED(Bind_Functions()))
        return E_FAIL;

    if (FAILED(Ready_Trigger()))
        return E_FAIL;

    // 중략

    return S_OK;
}
```

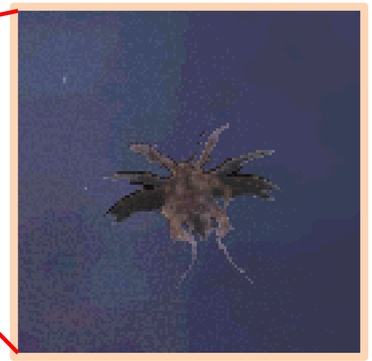
보스 몬스터의 속성 값을 대입하고 컴포넌트 모듈을 추가합니다.

공격 애니메이션에 진행될 때 호출되어야 할 함수들을 정의했습니다.

구역을 정의해서 보스 몬스터의 이동 경로를 정의했습니다.
보스 몬스터는 플레이어의 위치에 따라 정해진 경로로 이동합니다.

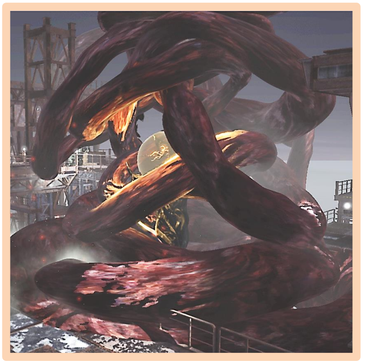
```
void CSaddlerObject::Priority_Update(_float fTimeDelta)
{
    if (true == m_bDeadEnd)
    {
        m_pGameInstance->Add_Timer(CMyUtils::Convert_String(m_szGameObjectTag), 0.f, [&]() {
            CSaddlerObjectSecond* pSaddlerObjectSecond = m_pGameInstance->Get_ObjectByName<CSaddlerObjectSecond>
(m_eCurLevel, TEXT("Layer_Monster"), TEXT("SaddlerObjectSecond"));
            if (nullptr != pSaddlerObjectSecond)
                pSaddlerObjectSecond->Initialize_PoolingObject(nullptr);
            }, false);

        m_pGameInstance->Add_Timer(CMyUtils::Convert_String(m_szGameObjectTag), 0.f, [&]() {
m_pGameInstance->Remove_Object(this); }, false);
    }

  // 중략
    __super::Priority_Update(fTimeDelta);
}
```

보스 몬스터가 사망 처리가 완료되면 파이널 페이즈를 활성화합니다.



<보스 몬스터 소환 패턴 예시>

<소환된 몬스터 예시>



<보스 몬스터 페이즈 전환 예시>

<일반 몬스터 예시 1>

<일반 몬스터 예시 2>

<일반 몬스터 예시 3>

<일반 몬스터 예시 4>

<일반 몬스터 예시 5>

<일반 몬스터 예시 6>

<일반 몬스터 예시 7>

<일반 몬스터 예시 8>

일반 몬스터 구현

기술 정의

일반 몬스터 클래스 구현

사용 목적

일반 몬스터 간의 중복 코드를 최소화하고
재사용성을 높이기 위해 공통 로직을 포함한 범용
몬스터 클래스로 사용했습니다.

구현 방법

몬스터 속성 구조체를 정의하고 몬스터 생성 시
정의된 구조체에 값을 담아 전달해서 몬스터
속성을 대입합니다.
몬스터 타입에 따라 정의된 모델과 FSM을
사용해서 몬스터는 FSM을 업데이트하고,
델리게이트 기반으로 이벤트를 처리하도록
구현했습니다.

```
HRESULT CMonsterCommonObject::Initialize(void* pArg)
{
    MONSTERCOMMONOBJECT_DESC* pDesc =
reinterpret_cast<MONSTERCOMMONOBJECT_DESC*>(pArg);
    // 중략

    _wstring strOri = pDesc->szGameObjectTag;
    switch (pDesc->eModelForm >= FORM_END ? (pDesc
->eModelForm % FORM_END) : pDesc->eModelForm)
    {
    case FORM_NORMAL_BARE:
        strOri += TEXT("Enemy_Common_GanadoA");
        pDesc->fHealth = 2000.f;
        pDesc->iAttack = 5.f;
        // 중략
    default:
        break;
    }
    lstrcpy(pDesc->szGameObjectTag, strOri.c_str());
    // 중략
    m_bEnemy = true;

    if (FAILED(__super::Initialize(pDesc)))
        return E_FAIL;

    if (FAILED(Ready_Components()))
        return E_FAIL;
    // 중략
    if (FAILED(Ready_TriggerHands()))
        return E_FAIL;

    m_pVariableContext->Add_Variable<_int>
("StateKey", (_int)(CHARST_IDLE));
    // 중략

    if (FAILED(Bind_Delegate()))
        return E_FAIL;

    if (FAILED(Ready_FSM()))
        return E_FAIL;
    // 중략
    return S_OK;
}
```

몬스터 타입에 따른 속성 구조체의 속성 값을 대입합니다.

몬스터가 사용하는 컴포넌트 모듈을 추가합니다.

몬스터 타입별로 미리 정의해둔 충돌체를 불러와서 생성합니다.

델리게이트를 호출할 변수를 선언하고 변수 변경을 감지해서 호출될 델리게이트를 정의했습니다

몬스터 타입별로 미리 정의된 FSM을 불러와서 생성합니다.

<충돌체 예시 1>

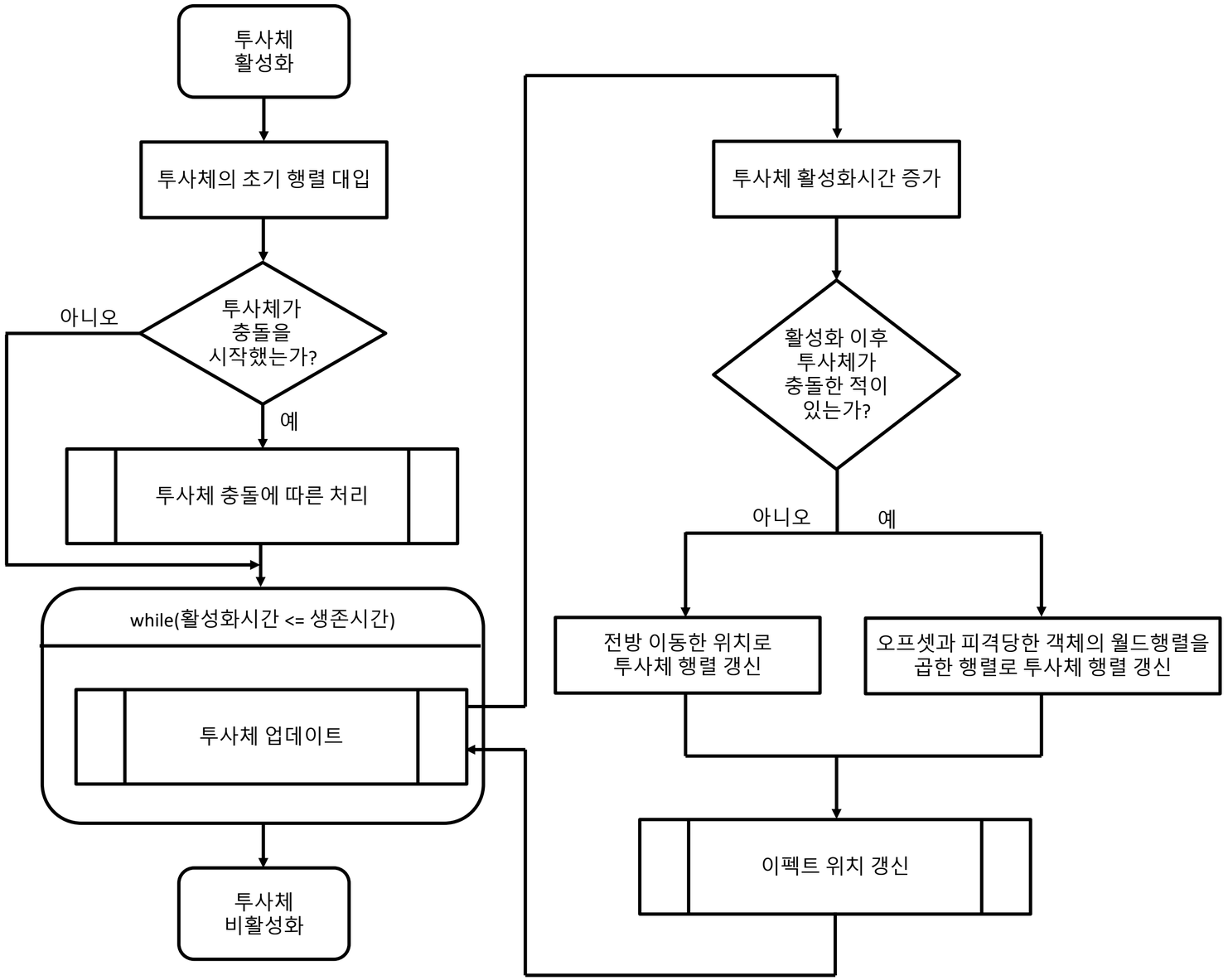
<충돌체 예시2>

<몬스터 타입별 FSM 예시 1>

<몬스터 타입별 FSM 예시 2>

<몬스터 타입별 FSM 예시 3>

<몬스터 타입별 FSM 예시 4>



<투사체 클래스 순서도>

투사체 구현

기술 정의

투사체용 클래스 구현

사용 목적

투사체가 발사되고 객체가 피격당하면 투사체가 객체를 따라 움직이고 피격 위치에 있도록 하기위해 사용했습니다.

구현 방법

투사체 객체를 만들고 비활성화 한 후, 투사체를 사용하는 객체에게 투사체의 포인터를 지니게해서 활성화하게 했습니다.
투사체를 지닌 객체에서 투사체 활성화 시 초기 값에 사용될 행렬을 전달하고 투사체 객체에서 초기 값을 대입하고 생존시간이 지나면 비활성화되도록 했습니다.

```
void CMonsterCommonObject::Set_Shot()
{
    if (nullptr == m_pMyBolt)
        return;

    CMonsterBolt::BOLTPOOLING PoolingDesc = {};

    PoolingDesc.WorldMatrix
= XMMatrixTranslation(m_fBoltOffset.x, m_fBoltOffset.y, m_fBoltOffset.z)
* XMLoadFloat4x4(m_pCompositeArmature->Get_SocketMatrixPtr("", "L_Wep"))
* m_pTransformCom->Get_WorldMatrix();

    if (FAILED(m_pMyBolt->Initialize_PoolingObject(&PoolingDesc)))
        return;

    m_pMyBolt->Set_Sleep(false);
}
```

투사체 포인터를 지닌
객체에서
투사체의 초기 행렬을
만들고 전달합니다

투사체를
활성화합니다.

<투사체 활성화 예시>

```
void CMonsterBolt::Update(_float fTimeDelta)
{
    m_fShotAfterTimer += fTimeDelta;

    if (true == m_bHideTimer)
    {
        m_fTimer += fTimeDelta;

        if (nullptr != m_pTargetMatrix)
            m_pTransformCom->Set_WorldMatrix(XMLoadFloat4x4(&m_OffsetMatrix)
* XMLoadFloat4x4(m_pTargetMatrix));

        if (m_fLifeTime <= m_fTimer)
        {
            m_pGameInstance->Add_Timer("OverLifeTime", 0.f, [&]() {
                Set_Sleep(true);
                });
        }
    }
    else
        m_pTransformCom->Go_Straight(fTimeDelta);

    if (4.f <= m_fShotAfterTimer && false == m_bHideTimer)
        m_bHideTimer = true;

  // 중략

}
```

투사체의 위치를
갱신합니다.

투사체의 활성화된 시간이
투사체의 생명 주기를
넘으면 투사체를
비활성화합니다.

<이동에 따른 투사체 위치 갱신 예시 1>

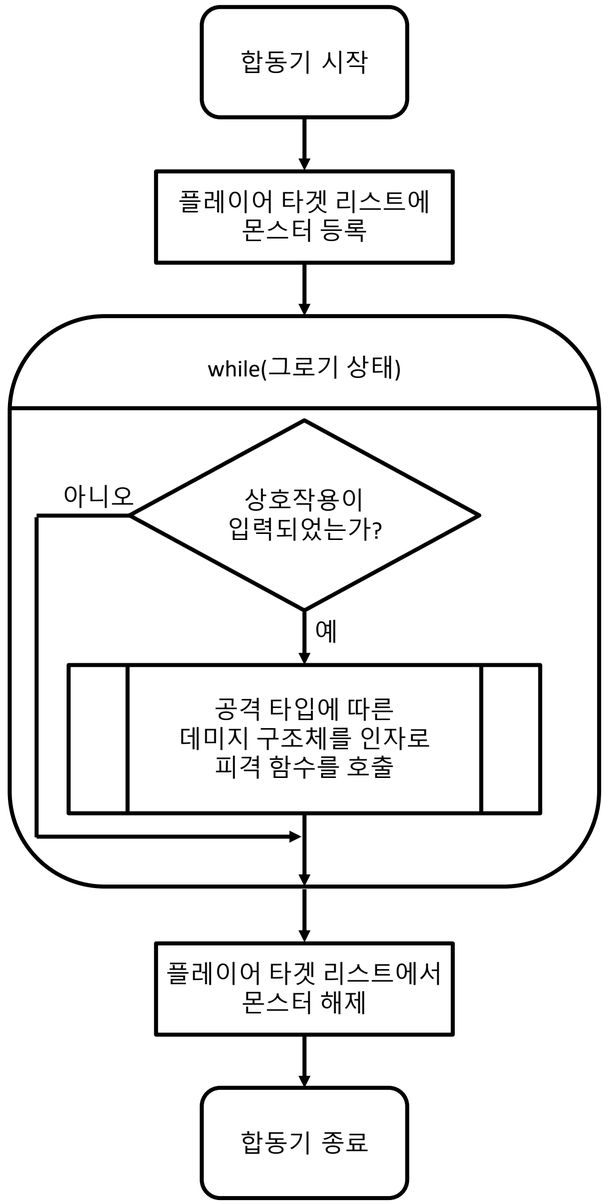
< 이동에 따른 투사체 위치 갱신 예시 2 >

<피격 대상에 따른 투사체 위치 갱신 예시 1>

<피격 대상에 따른 투사체 위치 갱신 예시 2>

<합동기 예시 1 – 몬스터의 잡기>

<합동기 예시 2 – 플레이어의 탈출>



<합동 애니메이션 순서도>

합동 애니메이션

기술 정의

플레이어와 몬스터가 상호작용해서 연계되는
애니메이션 진행

사용 목적

몬스터와 플레이어가 동시에 참여하는
애니메이션의 연출을 구현하기 위해
사용했습니다.

구현 방법

합동기 애니메이션을 먼저 진행하는 객체가 다른
객체에게 공격 타입과 합동기 타입이 정의된
데미지구조체를 인자로 피격 함수를 호출하고,
피격된 객체가 전달받은 합동기 타입에 맞춰
일관된 애니메이션을 재생하도록 구현했습니다.

플레이어 주도 합동기 애니메이션 진행

<합동기 진행 예시 1 – 플레이어의 암살>

<합동기 진행 예시 2 – 플레이어의 뒤차기>

<합동기 진행 예시 3 – 플레이어의 백드롭>

```
HRESULT CMonsterCommonObject::AttackTriggerRegister_Callback(CTriggerSocket* pTriggerSocket)
{
  // 중략

            CPlayerCharacter* pPlayer = dynamic_cast<CPlayerCharacter*>(pBound->Get_Owner());

            if (nullptr == pPlayer)
                return;

            if (false == pPlayer->Can_Hit())
                return;

            m_bGrappling = true;
            DamageEventRE DmgEvtRE = { };
            DmgEvtRE.fDamage = m_iAttack;
            DmgEvtRE.eType = DAMAGE_NORMAL;
            DmgEvtRE.eReactionType = REACTION_JOINTATTACK;
            DmgEvtRE.eJointPoint = JOINT_HOLD;
            DmgEvtRE.ColInfo = *pInfo;

            _vector vMyLook = XMVector3Normalize(Set_ZeroYW(m_pTransformCom
->Get_State(CTransform::STATE_LOOK)));
            _vector vTargetLook = XMVector3Normalize(Set_ZeroYW(m_pTarget
->Get_TransformCom()->Get_State(CTransform::STATE_LOOK)));
            _vector vDot = XMVector3Dot(vMyLook, vTargetLook);
            _float fToFrontAngle = XMVectorGetX(vDot);
            if (0.f <= fToFrontAngle)
                DmgEvtRE.iUserData = (_int)JOINT_HOLD_TYPE::TYPE_BACK;
            else
                DmgEvtRE.iUserData = (_int)JOINT_HOLD_TYPE::TYPE_FRONT;

            pBound->Get_Owner()->Take_Damage(m_fAttack, &DmgEvtRE, this, this);

  // 중략
    });
```

다른 객체가 합동기 진행이
불가능하다면
함수를 종료합니다.

공격 타입과 합동기 타입 등
데미지 구조체를 정의합니다.

객체들의 정면 벡터를
내적해서 객체들의 정면
방향 일치여부를 대입합니다.

공격 타입과 합동기 타입이
정의된 데미지구조체를
인자로 다른 객체의
피격함수를 호출합니다.

< 합동기 진행 전 예시 >

< 합동기 진행 중 예시 1 >

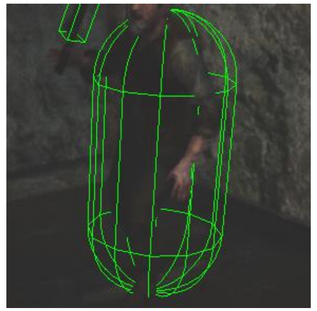
< 합동기 진행 중 예시 2 >

```
HRESULT CMonsterCommonObject::Ready_TriggerHands()
{
    vector<BONE_BIND> HitBindList =
CMyUtils::Load_BoneBind("../Bin/Resources/ColInfo/CommonMonster_Hit_Bind.yaml");

    m_iAtkBoundIndex = HitBindList.size();

    vector<BONE_BIND> AtkBindList;

    switch (m_eModelType)
    {
    case Client::CMonsterCommonObject::FORM_NORMAL_BARE:
        AtkBindList = CMyUtils::Load_BoneBind(
"../Bin/Resources/ColInfo/CommonMonster_Bare_Bind.yaml");
        break;
        // 중략
    }
    HitBindList.reserve(HitBindList.size() + AtkBindList.size());
    for (auto& BoneBind : AtkBindList)
        HitBindList.push_back(BoneBind);

    /* TriggerSocket */
    CTriggerSocket::TriggerSoket_DESC TriggerDesc{};
    TriggerDesc.fRadius = 1.f;
  // 중략
```

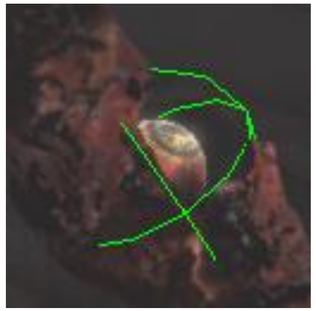
피격용 충돌체를 만들기 위한 충돌체 정보를 저장합니다.
이후의 충돌체 정보는 공격용 충돌체 정보이기 때문에 공격용 충돌체의 인덱스 시작번호를 저장합니다.

공격용 충돌체 정보를 저장하고 충돌체 정보를 한쪽에 모두 저장합니다.

<박스형 충돌체 예시>



<캡슐형 충돌체 예시>



<구체형 충돌체 예시>

충돌체 관리

기술 정의

용도에 따라 충돌체를 만들어서 관리

사용 목적

피격용 충돌체와 공격용 충돌체를 구분하고, 충돌체를 기준으로 피격 부위를 판별하기 위해 사용했습니다.

구현 방법

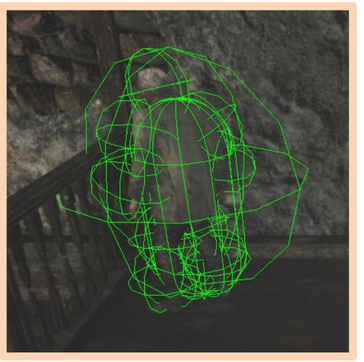
충돌체의 크기와 위치 정보 등을 미리 정의해서 용도에 따라 별개의 파일에 저장하고 불러와서 사용했습니다.
충돌체의 용도에 따라 동작할 함수를 따로 바인딩했습니다.

```
HRESULT CMonsterCommonObject::Ready_TriggerHands()
{
  // 중략
    /* TriggerSocket */
    CTriggerSocket::TriggerSoket_DESC TriggerDesc{};
    TriggerDesc.fRadius = 1.f;
  // 중략

    for (_int i = 0; i < HitBindList.size(); ++i)
    {
      // 중략
        TriggerDesc.strBoneName = HitBindList[i].strBoneName;
        TriggerDesc.vOffsetTranslation = HitBindList[i].vTranslation;
        TriggerDesc.vOffsetRotation = HitBindList[i].vRotation;
        TriggerDesc.vScale = HitBindList[i].vRigidScale;
        TriggerDesc.eRigidType = HitBindList[i].eTypeRigid;
        TriggerDesc.iTriggerIndex = m_TriggerHands.size();

        CTriggerSocket* pSocket = dynamic_cast<CTriggerSocket*>(m_pGameInstance
->Clone_Prototype(PROTOTYPE::TYPE_GAMEOBJECT, LEVEL_STATIC
, TEXT("Prototype_GameObject_TriggerSocket"), &TriggerDesc));
        if (pSocket)
        {
            m_TriggerHands.push_back(pSocket);
            switch (HitBindList[i].eTypeRigid)
            {
            case Engine::RIGIDSHAPE::CAPSULE:
                if (i < m_iAtkBoundIndex)
                    HitTriggerRegister_Callback(pSocket);
                else
                    AttackTriggerRegister_Callback(pSocket);
                break;
                // 중략
            default:
                break;
            }
            if (i >= m_iAtkBoundIndex)
                m_TriggerIndex.emplace(TriggerDesc.strBoneName
, m_TriggerHands.size() - 1);
        }
    }
    return S_OK;
}
```

저장된 충돌체 정보를 충돌체 정보 구조체에 대입합니다.

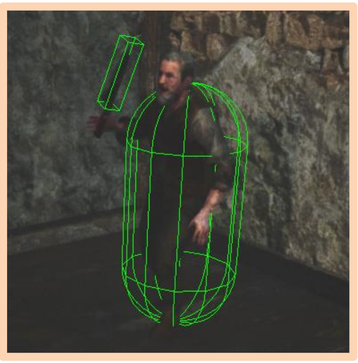
충돌체 정보 구조체로 충돌체를 생성합니다.

충돌체의 용도에 따라서 충돌 처리 함수를 바인딩합니다.
공격용 충돌체는 공격 애니메이션이 진행될 때 해당 위치의 충돌체가 활성화됩니다.

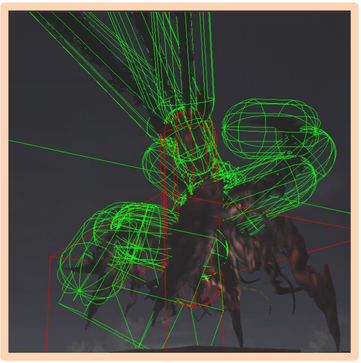


<피격용 충돌체 예시1>

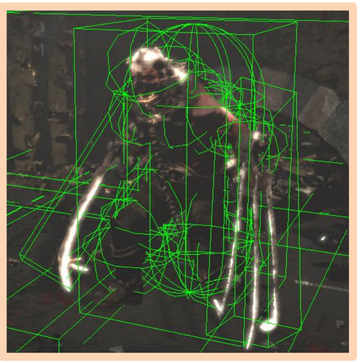
<피격용 충돌체 예시2>



<공격용 충돌체 예시1>



<공격용 충돌체 예시2>



<충돌체 전체 예시>

```
    m_pVariableContext->Bind_Delegate("HitNorKey", [&]() {
        DamageType eHitTtype = (DamageType)m_pVariableContext
->Get_Variable<_int>("HitTypeKey");
      // 중략
        else if(DamageType::DAMAGE_RAY == eHitTtype)
        {
            CMeshSprite::MeshSprite_DESC MeshSpritedesc;
          // 중략
            _vector vBloodUp =
XMVector3Normalize(XMVectorSetW(XMVectorNegate(m_pVariableContext
->Get_Variable<_vector>("HitNorKey")), 0.f));

            _matrix matBloodWorld = {};
          // 중략
            matBloodWorld.r[1] = XMVectorSetW(vBloodUp, 0.f) *
MeshSpritedesc.vScale.y;

            _matrix InvWorld = XMMatrixInverse(nullptr, m_pTransformCom-
>Get_WorldMatrix());
            XMStoreFloat4x4(&MeshSpritedesc.OffsetMatrix, matBloodWorld*
InvWorld);
            MeshSpritedesc.pTargetWorldMatrix = m_pTransformCom
->Get_WorldMatrix_Ptr();
            if (FAILED(m_pGameInstance->Add_GameObject(LEVEL_STATIC,
TEXT("Prototype_GameObject_BossVFX09"),
                m_eCurLevel, TEXT("Layer_Effect"), &MeshSpritedesc)))
                return;

            _matrix matColDir = XMMatrixTranslationFromVector(m_pVariableContext
->Get_Variable<_vector>("HitPosKey"));

            matColDir.r[0] = vBloodRight;
          // 중략
            XMStoreFloat4x4(&m_BloodParticleLocalOffset, matColDir* InvWorld);

            _uint iIndex = { };
            _string strBoneName = m_pVariableContext->Get_Variable<_string>
("BoneNameKey");
          // 중략

            m_BloodParticleOffset = matColDir;

            if (nullptr == m_pBloodParticleEmitter)
            {
                CParticleEmitter* pEmitter = { nullptr };
                m_pGameInstance->Spawn_Particle(L"Blood_Out", pEmitter);
                m_pBloodParticleEmitter = pEmitter;
            }
            else
            {
                m_pBloodParticleEmitter->Replay();
            }
        }});
```

피격 방향을 이용해 이펙트를 회전시키고 피격 위치에서 이펙트가 출력되도록 이펙트 행렬을 구성합니다.

이펙트 위치를 갱신하기 위한 오프셋을 만들고 메시 이펙트를 생성합니다.

메시 이펙트 행렬을 기반으로 파티클 이펙트 행렬을 만들고 파티클 이펙트를 재생합니다.

<피격 블러드 이펙트 및 메시 이펙트 예시>

이펙트 출력

기술 정의

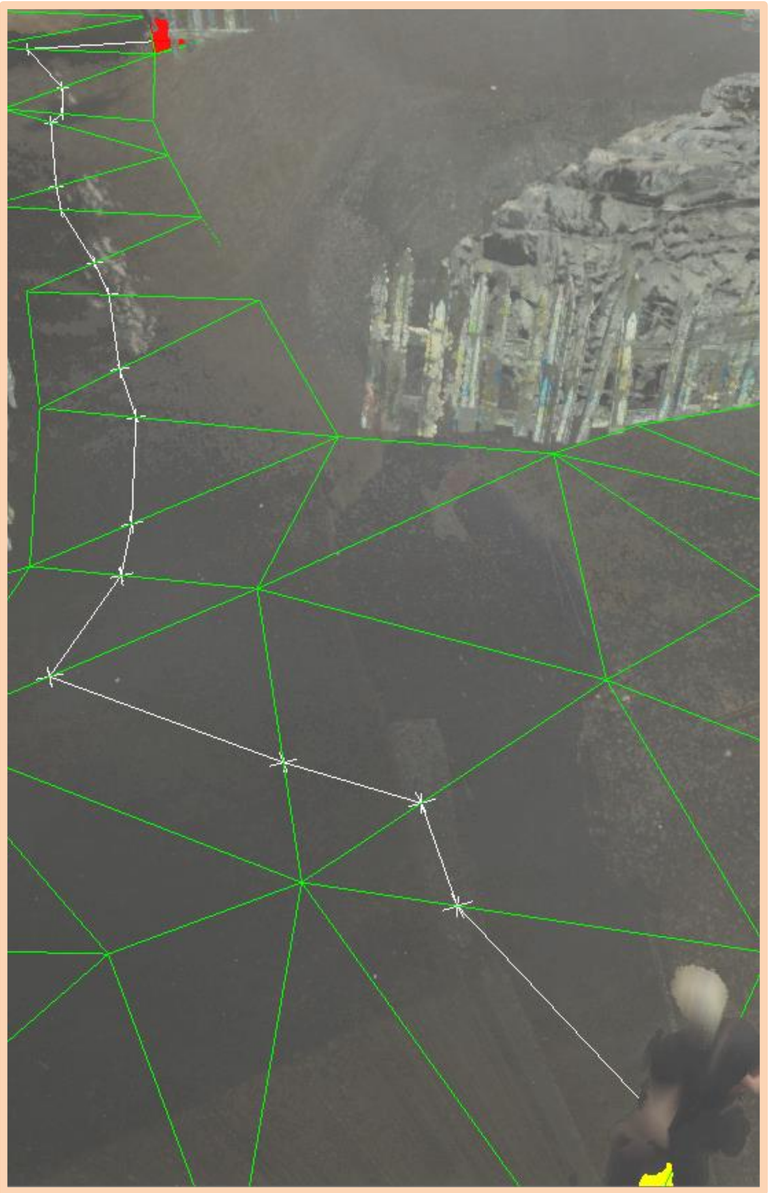
파티클 이펙트, 메시 이펙트, 스크린 스페이스 데칼 등의 이펙트 출력 위치를 결정

사용 목적

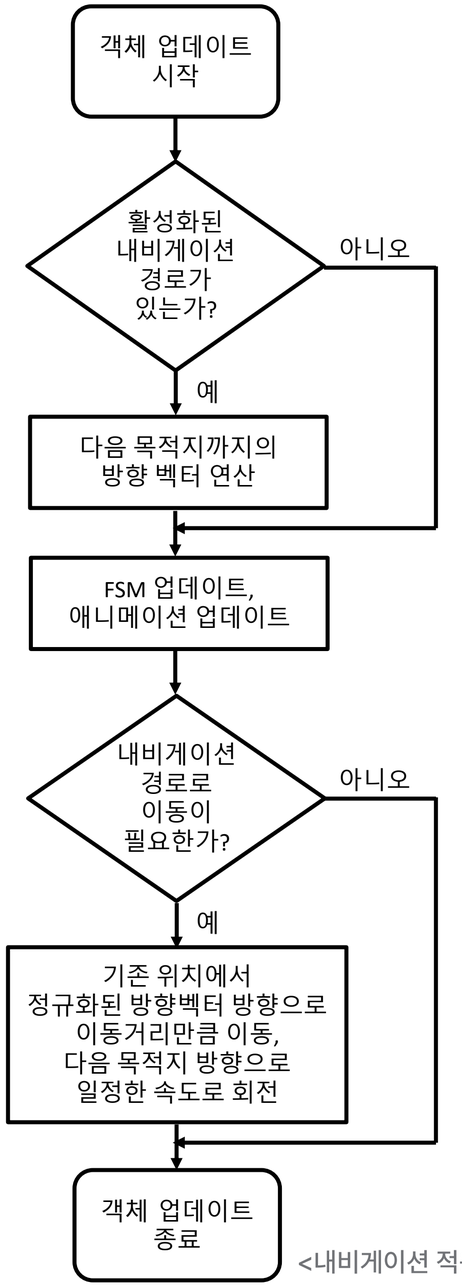
이펙트마다 생성되는 위치를 결정하기위해 사용했습니다.

구현 방법

이펙트마다 생성 위치를 다르게 했습니다.
파티클 이펙트는 오프셋 행렬과 특정 본의 로컬 행렬, 그리고 객체의 월드 행렬의 곱으로 좌표를 지정했습니다.
피격 시 발생하는 이펙트는 충돌 지점과 피격 방향벡터의 역벡터를 활용해서 이펙트 위치와 이펙트 출력 방향을 갱신했습니다.
스크린 스페이스 데칼은 충돌 지점에서 피격 방향 벡터로 레이 캐스트를 진행해서 인접한 지형지물에 생성했습니다.



<내비게이션 경로 예시>



<내비게이션 적용 순서도>

내비게이션 적용

기술 정의

내비게이션 경로를 따른 루트 애니메이션 이동

사용 목적

내비게이션을 통해 이동 경로가 정해지더라도 루트 애니메이션이 적용되어 이동 값은 애니메이션에 종속되어 이동 시 어색함이 있어 이를 분리해 내비게이션을 따른 자연스러운 이동을 적용하기 위해 사용했습니다.

구현 방법

현재 위치를 저장하고 내비게이션 경로에 따른 다음 목적지까지의 방향 벡터를 구합니다. 애니메이션을 진행시켜 이동거리를 구하고 방향 벡터와 이동거리를 곱해서 내비게이션에 따른 이동된 좌표를 구하고, 다음 목적지가 정면이 되도록 서서히 회전시켰습니다.

```
void CMonsterCommonObject::Update(_float fTimeDelta)
{
  // 중략
    _vector vCurrentPos = m_pTransformCom->Get_State(CTransform::STATE_POSITION);
    _vector vDirection = {};
  // 중략
    _vector vTargetPos = m_pTarget->Get_TransformCom( )
->Get_State(CTransform::STATE_POSITION);
    _vector vNextPos = {};
  // 중략
    _bool bIndexNav = m_pNavigationCom->Set_IndexByNav(m_pTransformCom);
    m_pNavigationCom->Update(fTimeDelta);

    if (m_bChasing && m_pNavigationCom->Request_Path_Async(vCurrentPos, vTargetPos))
    {
        if (m_pNavigationCom->Get_NextWayPoint(vCurrentPos, vNextPos))
            m_bFollowNav = true;
    }
    else
        m_bFollowNav = false;

    for (auto& Pair : m_EasyFSMs)
        Pair.second->Update(fTimeDelta);
    m_pCompositeArmature->Update_Animation(fTimeDelta);

    if (true == m_bFollowNav)
    {
        _vector vDirectionNor = XMVector3Normalize(Set_ZeroYW(vDirection));
        _vector vUpdatePos = m_pTransformCom->Get_State(CTransform::STATE_POSITION);
        _float fUpdateDistance = XMVectorGetX(XMVector3Length(vUpdatePos
- vCurrentPos));
        m_pTransformCom->Set_State_Notify(CTransform::STATE_POSITION, vCurrentPos
+ vDirectionNor * fUpdateDistance);

      _vector vLook = XMVector3Normalize(Set_ZeroYW(m_pTransformCom
->Get_State(CTransform::STATE_LOOK)));
      _float fAngle = XMVectorGetX(XMVector3Dot(vLook, vDirectionNor));
      m_pTransformCom->Rotation_Dir_Smooth(vDirectionNor, fTimeDelta * 4.f);
    }

    __super::Update(fTimeDelta);
}
```

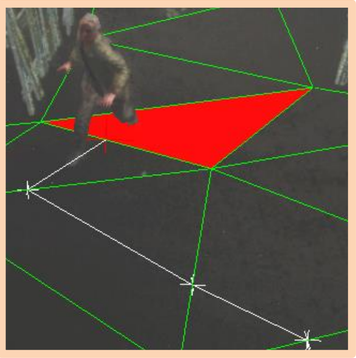
현재 위치에 내비게이션 셀이 있는지 검사하고 경로 타이머를 누적합니다.

목표까지 내비게이션 경로가 있는지 확인하고 없다면 경로를 생성합니다.

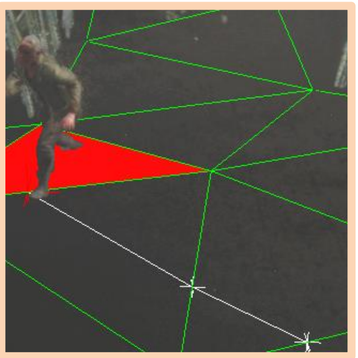
내비게이션 경로의 다음 포인트를 탐색합니다.

다음 포인트 방향으로 좌표를 변경합니다.

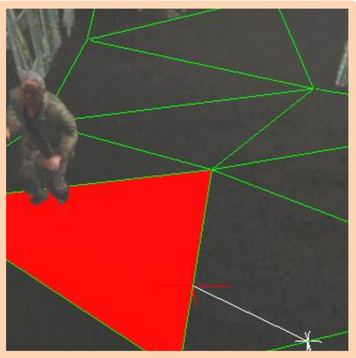
다음 포인트 방향으로 서서히 회전합니다.



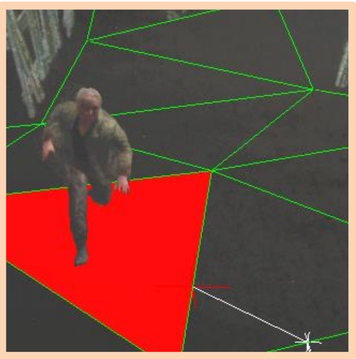
<내비게이션 적용 예시 1>



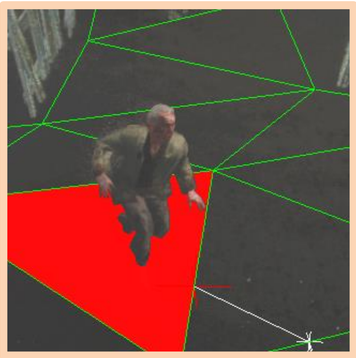
<내비게이션 적용 예시 2>



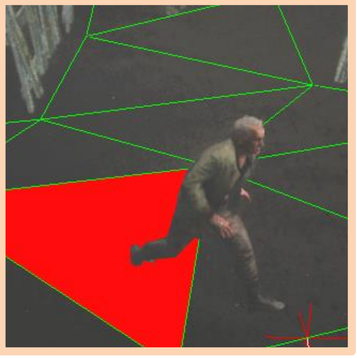
<내비게이션 적용 예시 3>



<내비게이션 적용 예시 4>



<내비게이션 적용 예시 5>



<내비게이션 적용 예시 6>

DirectX9을 이용한 2.5D 팀 포트폴리오

원작게임

IRA

작업기간

2024.11.08 ~ 2024.12.17

작업인원

4인 게임 모작

담당 업무

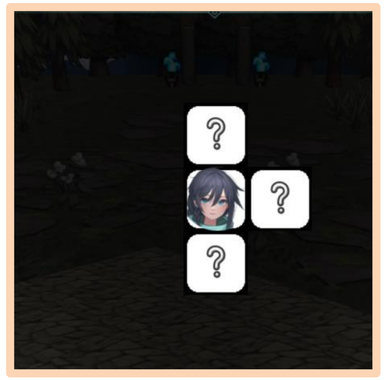
맵 구현 담당

사용 언어

C++

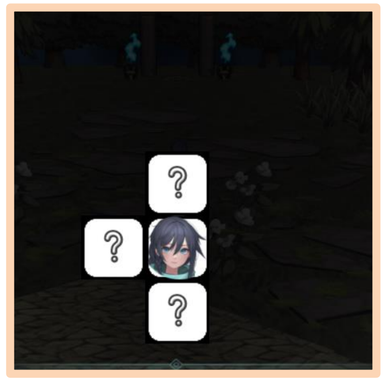
제작 환경

Visual Studio 2022
IMGUI

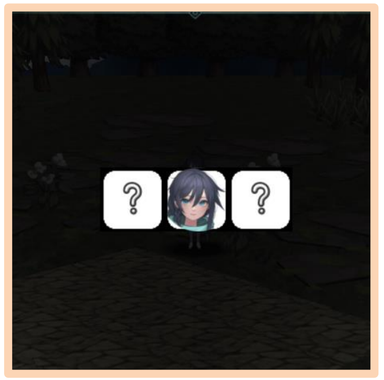


<랜덤 맵 생성 예시 1 >

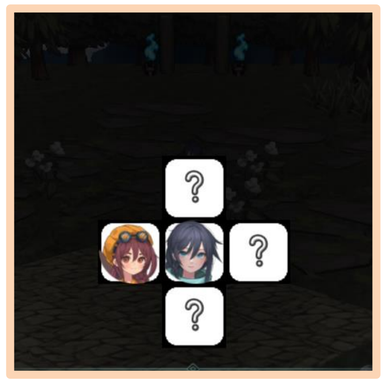
< 랜덤 맵 생성 예시 2 >



< 랜덤 맵 생성 예시 3 >



< 랜덤 맵 생성 예시 4 >



< 랜덤 맵 생성 예시 5 >

< 랜덤 맵 생성 예시 6 >

던전형 맵

기술 정의

정의해둔 방 정보를 랜덤하게 배치해서 임의의 던전을 만드는 기술

사용 목적

만들어둔 방을 랜덤하게 배치해서 던전처럼 만들기위해서
사용했습니다.

구현 방법

가상의 10*10 이차원 배열을 두고 중간 번호로 방을 만듭니다.
만들어진 번호를 기준으로 4방향 중 랜덤한 방향으로 생성 가능한
번호에 방을 새로 만들고 이웃한 방 번호들을 추가합니다.

```
HRESULT CRandomRoomMgr::GenerateRooms(_int iStageNum, map<_int, RMINFO>* pMapRooms)
{
    srand(static_cast<unsigned int>(time(NULL)));

    const _int  iStartRoomNum               = 55;
    const _int  arrDirection[4]             = { -1, 1, -10, 10 };
  // 중략
    _int        arrNeighborRoomNums[100];
    RMINFO      tRoomInfo;

    do
    {
      // 중략
        pMapRooms->insert({ iStartRoomNum, tRoomInfo });

        queue<_int> roomQueue;
        roomQueue.push(iStartRoomNum);
        arrRoomOccupied[iStartRoomNum] = true;
        AddNeighbors(iStartRoomNum, arrNeighborRoomNums);

        roomQueue.push(iStartRoomNum);

        while (!roomQueue.empty())
        {
            if (pMapRooms->size() == arrStageRoomNums[iStageNum]) break;
            _int iNow = roomQueue.front();
            roomQueue.pop();

            for (int i = 0; i < 4; ++i)
            {
                if (pMapRooms->size() == arrStageRoomNums[iStageNum]) break;
                _int iNext = iNow + arrDirection[i];

                if (IsInside(iNext) == false)               continue;
                else if (arrRoomOccupied[iNext])            continue;
              // 중략
                else
                {
                    ZeroMemory(&tRoomInfo, sizeof(tRoomInfo));

                    tRoomInfo.bRoomClear = false;
                    tRoomInfo.eRoomType = RM_NORMAL;
                    tRoomInfo.iObjectNum = 0;
                    tRoomInfo.iRoomNum = iNext;
                    pMapRooms->insert({ iNext, tRoomInfo });

                    roomQueue.push(iNext);
                    arrRoomOccupied[iNext] = true;
                    AddNeighbors(iNext, arrNeighborRoomNums);
                }
            }
        }
    } while (pMapRooms->size() != arrStageRoomNums[iStageNum]);
  // 중략
    return S_OK;
}
```

아무 정보가 없는 방을 만들고 초기화합니다.

인접한 방들에 이웃을 추가합니다.

정해진 개수의 방을 생성하하고 이웃 설정을 합니다.

<생성된 전체 방 구조 예시 1>

<생성된 전체 방 구조 예시 2>

<대쉬 애니메이션>

<대쉬 애니메이션>

<루트 애니메이션 미적용 시 대쉬 후 위치>

<루트 애니메이션 적용 시 대쉬 후 위치>

<루트 애니메이션 미적용 시 대쉬 후 위치>

<루트 애니메이션 적용 시 대쉬 후 위치>

맵 오브젝트 구현

기술 정의

맵에 배치되어 다른 객체의 이동을 방해하거나 부서지고, 피해를 입히는 환경 요소 오브젝트

사용 목적

맵을 구성하는 환경 요소들을 배치하고, 환경 요소를 이용한 함정이나 이동을 방해하고, 맵을 장식하기 위해 사용했습니다.

구현 방법

환경 요소를 열거형으로 정의해서 맵 오브젝트 타입으로 사용했고,
환경 요소용 클래스를 만들어서 맵 오브젝트 타입에 따라
맵 오브젝트가 다른 텍스쳐를 사용하도록 구현했습니다.
맵 오브젝트 타입별로 충돌 시 동작하는 함수를 다르게 했습니다.
환경 요소가 플레이어를 가리지 않도록 플레이어의 앞에 위치하면
알파 테스트를 적용했습니다.

```
void CEnvironmentObject::Set_EnvironmentTexture(ENVIRONMENTID _ePreEnvironmentID)
{
    Clear_Texture();

    IComponent* pComponent = nullptr;

    wstring wstrName = (g_pUtility->Get_DataMgrClass()
->Get_EnvironmentData((ENVIRONMENTID)(m_iEnvironmentID)))->szEnvironmentTextureTag;

    pComponent = m_pTexture = dynamic_cast<CTexture*>(g_pUtility
->Get_ComponentMgrClass()->Clone_Prototype(wstrName.c_str()));
    NULL_CHECK(pComponent);

    m_pTransform->Set_BillboardY(false);

    Set_Atk();

    switch (m_eEnvironmentID)
    {
    case EN_RUNTRAP3:
        m_pCollider->Set_Size(_vec3(0.75f, 0.75f, 0.75f));
        m_pTransform->Set_BillboardY(true);
        break;
        // 중략
    default:
        break;
    }

    if (_ePreEnvironmentID >= EN_DESTROYSTRUCT && _ePreEnvironmentID < EN_TRAP)
    {
        _vec3 vPos = m_pTransform->Get_Info(INFO_POS);
        vPos.y = m_pTexture->Get_Height() / 80.f;
        m_pTransform->Set_Position(vPos);
    }
    m_mapComponent[ID_STATIC].insert({ L"Com_Texture", pComponent });
}
```

텍스쳐 컴포넌트를 할당 해제합니다.

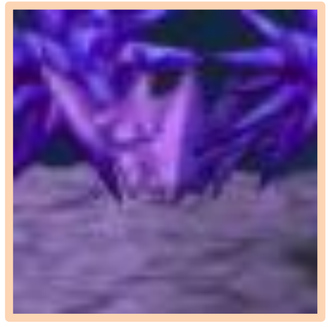
맵 오브젝트 타입에 따른 텍스쳐 이름을 저장하고,
저장된 텍스쳐 이름으로 텍스쳐 컴포넌트를 할당합니다.

맵 오브젝트 타입에 따라 함정 오브젝트들의
피해량과 충돌체 크기 등 속성 값을 대입합니다.

< 파괴 전 환경 오브젝트 예시 >

< 파괴 후 환경 오브젝트 예시 >

< 함정용 환경 오브젝트 예시 1>



< 함정용 환경 오브젝트 예시 2 >

< 발사용 환경 오브젝트 예시 >

```
_int CEnvironmentObject::Update_GameObject(const _float& fTimeDelta)
{
    if (m_fDeadTimer > 0.f)
    {
        m_fDeadTimer -= fTimeDelta;

        if (m_iObjectNum % 2)
            m_pTransform->Set_Position(m_pTransform->Get_Info(INFO_POS)
+ (_vec3(-5.f, 0.f, 5.f) * -fTimeDelta));
        else
            m_pTransform->Set_Position(m_pTransform->Get_Info(INFO_POS)
+ (_vec3(-5.f, 0.f, 5.f) * fTimeDelta));

        if (m_fDeadTimer <= 0.f)
            m_bDead = true;
    }

    if (m_bDead) return UD_DEAD;
  // 중략
}
```

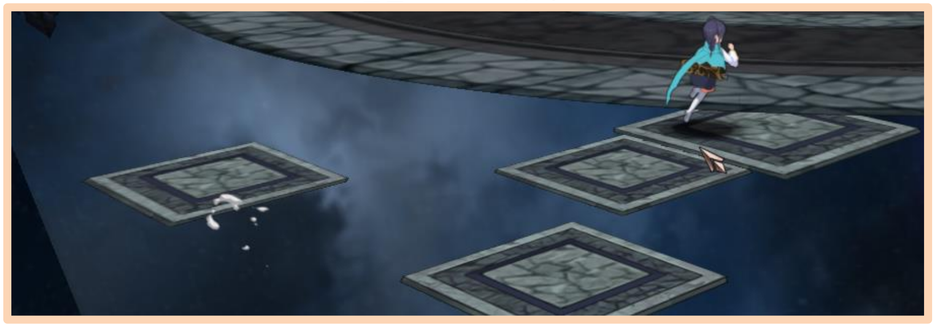
움직이는 환경 오브젝트만 데드 타이머를 양수로 설정하고 데드 타이머를 줄여갑니다.

발판이 다른 발판에 겹쳐지지 않는 왼쪽 상단이나 오른쪽 하단 방향으로 이동되게 합니다.

일정 시간이 지나면 발판이 사라지게 합니다.

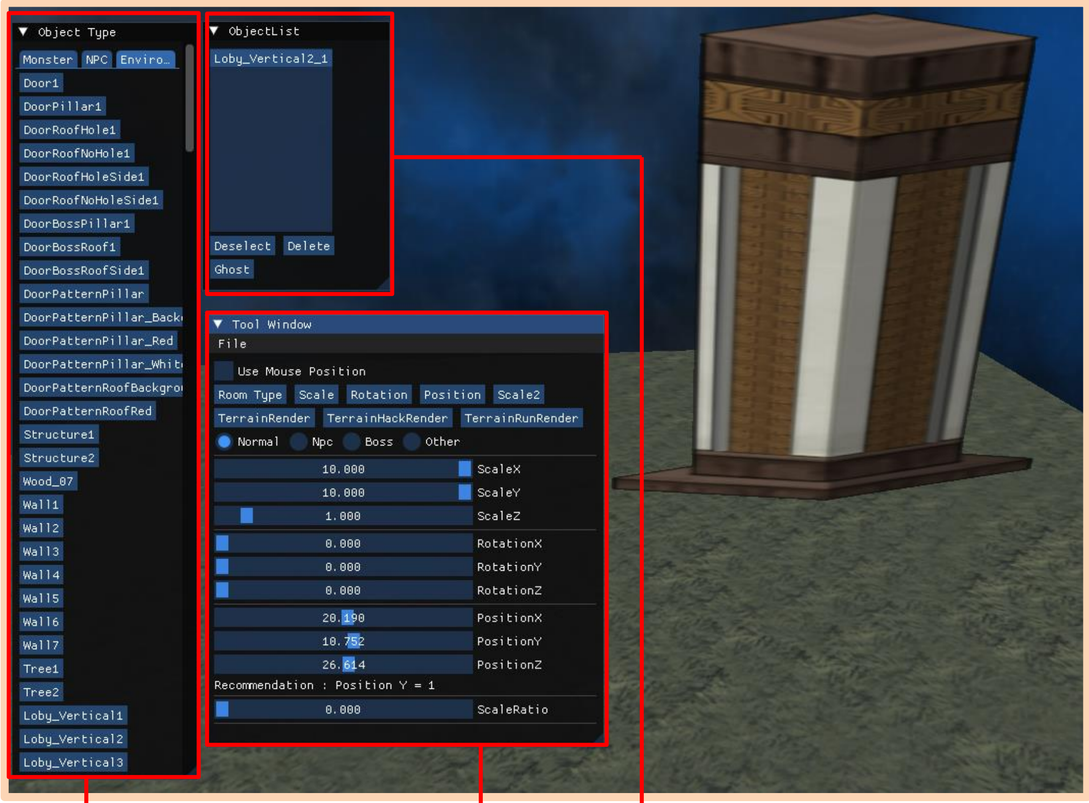
< 발판 오브젝트 예시 1 >

< 발판 오브젝트 예시 2 >



< 발판 오브젝트 예시 3 >

< 발판 오브젝트 예시 4 >



<맵 툴>

배치할 오브젝트를 선택합니다.

배치된 오브젝트를 표시하고, 출력합니다.

오브젝트의 위치, 크기, 회전 등의 속성 값을 출력합니다.

맵 툴

기술 정의

오브젝트를 배치할 수 있는 툴 구현

사용 목적

맵에 오브젝트를 배치하고 맵 정보를 저장해서 사용하기 위해 맵 툴을 만들었습니다.

구현 방법

IMGUI를 이용해 툴에서 오브젝트의 속성 값을 선택하고, 오브젝트를 배치할 수 있도록 했습니다.
현재 배치된 오브젝트들을 파일로 저장할 수 있도록 했고, 해당 파일을 불러와서 사용하거나 수정할 수 있도록 했습니다.

```
void CImGuiManager::Render_MapTool()
{
  // 중략
    if (m_pSelectObject)
    {
        m_pSelectObject->Set_ObjectID(m_tRoomInfo.eObjID);
        m_pSelectObject->Set_EnvironmentID(m_tRoomInfo.iEnvironmentID);
        if (m_tRoomInfo.eObjID != Engine::OBJ_ENVIRONMENT)
            dynamic_cast<CMapObject*>(m_pSelectObject)->Set_StructTexture();
        CTransform* pTransform = dynamic_cast<CTransform*>(m_pSelectObject
->Get_Component(ID_DYNAMIC, L"Com_Transform"));
        pTransform->Set_Scale(m_tRoomInfo.vScale);
        pTransform->Set_Rotation(D3DXToRadian(m_tRoomInfo.vRotation));
        pTransform->Set_Position(m_tRoomInfo.vPos);
    }
    else if (bUsingMousePos)
    {
        CTransform* pMouseTransform = nullptr;
        CTexture* pMouseTexture = nullptr;

        pMouseTransform = dynamic_cast<Engine::CTransform*>(m_pDebugGameObject
->Get_Component(ID_DYNAMIC, L"Com_Transform"));
        pMouseTexture = dynamic_cast<Engine::CTexture*>(m_pDebugGameObject
->Get_Component(ID_STATIC, L"Com_Texture"));

        pMouseTransform->Set_Scale(m_tRoomInfo.vScale);
        pMouseTransform->Set_Rotation(D3DXToRadian(m_tRoomInfo.vRotation));
        m_tRoomInfo.vPos.x = pMouseTransform->Get_Info(INFO_POS).x;
        m_tRoomInfo.vPos.z = pMouseTransform->Get_Info(INFO_POS).z;
        pMouseTransform->Set_Position(m_tRoomInfo.vPos);
    }
    else
    {
        CTransform* pMouseTransform = nullptr;
        CTexture* pMouseTexture = nullptr;

        pMouseTransform = dynamic_cast<Engine::CTransform*>(m_pDebugGameObject
->Get_Component(ID_DYNAMIC, L"Com_Transform"));
        pMouseTexture = dynamic_cast<Engine::CTexture*>(m_pDebugGameObject
->Get_Component(ID_STATIC, L"Com_Texture"));

        pMouseTransform->Set_Scale(m_tRoomInfo.vScale);
        pMouseTransform->Set_Rotation(D3DXToRadian(m_tRoomInfo.vRotation));
        pMouseTransform->Set_Position(m_tRoomInfo.vPos);
    }
  // 중략
}
```

선택한 오브젝트가 있는 경우에
오브젝트의 속성 값을
툴에서 입력한 속성 값으로 변경합니다.

마우스 위치를 속성 값으로 사용하려는 경우에
툴에 입력된 위치 값을
마우스의 위치 값으로 변경합니다.

나머지 모든 경우에 툴에 입력된 값을
다음에 추가될 오브젝트의 속성 값으로 변경합니다.

< 툴 이미지 예시 – 오브젝트 선택 >

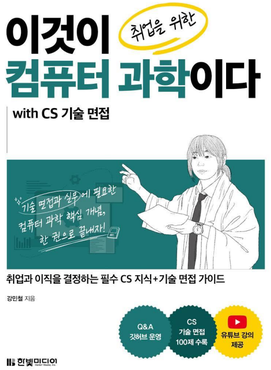
< 툴 이미지 예시 – 마우스 위치 >

< 툴 이미지 예시 – 미선택 >

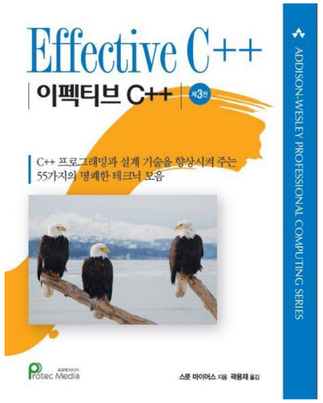
BIBLIOGRAPHY

참고문헌

• 이것이 자료구조+알고리즘이다 with C 언어



• 이것이 취업을 위한 컴퓨터 과학이다 with CS 기술 면접



• Effective C++

2026

한 걸음 한 걸음 꾸준히 나아가며
확실한 성장을 이루겠습니다.

이름 김창홍

전화번호 010-4720-0578

이메일 1138aiie@gmail.com